Java Full Stack Development

2. REST API Design and Error Models

Module Topics

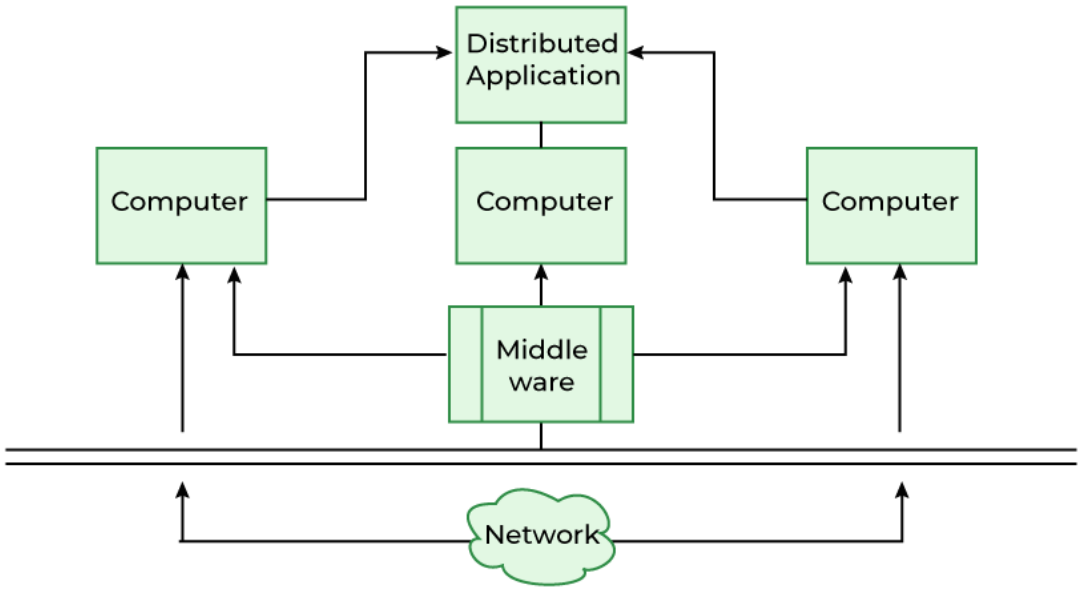
- The REST Solution
- Resource-Oriented URL Design
- HTTP Status Code Discipline
- Standard Error Payload Design
- API Versioning Strategy
- Idempotency and Safe Methods

The REST Solution

© 2026 by Innovation In Software Corporation

Distributed Systems

- The distributed systems problem
  - Building software that runs on a single machine is a solved problem
  - The hard problem is building software where components run on different machines and must communicate reliably
  - Late 1990s
    - Applications were being decomposed across servers
    - Data lived in one place, business logic in another, user interfaces in a third
    - The internet itself was a global network of machines that needed to exchange information without any central coordinator
  - This created three concrete problems that had to be solved

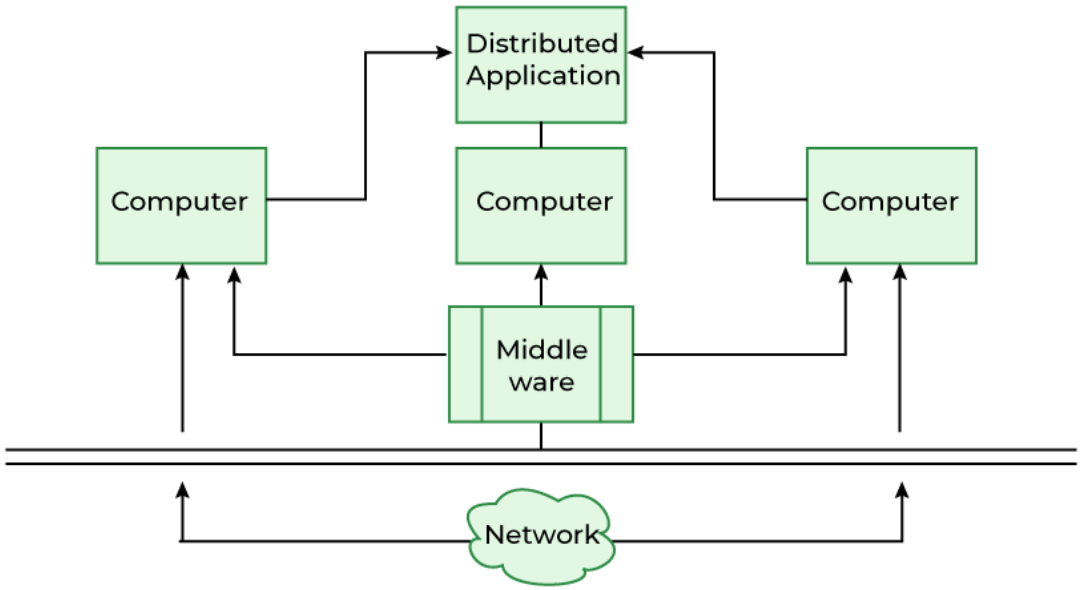


© 2026 by Innovation In Software Corporation

Image Credit: https://www.geeksforgeeks.org/operating-systems/types-of-distributed-system/

Distributed Systems

- Discovery
  - How does a component discover what another component can do?
  - Service A needs to call service B but needs to know
    - What operations B supports
    - What data format it expects, and what B will return
  - Without a shared convention, every integration requires customized knowledge
  - Scaling discovery to hundreds of service with individualized interfaces becomes a significant engineering roadblock



© 2026 by Innovation In Software Corporation

Image Credit: https://www.geeksforgeeks.org/operating-systems/types-of-distributed-system/

Distributed Systems

- Infrastructure support
  - Network infrastructure
    - Load balancers, caches, proxies, and gateways
  - Connects components by routing requests, caching responses and retrying failures
    - Needs to understand what is happening at the application level
  - If every application uses a different protocol, infrastructure is blind as to how to provide services
  - It is reduced to only moving bytes as a transport layer
    - All of the routing, caching and other services must now be handled at the application layer

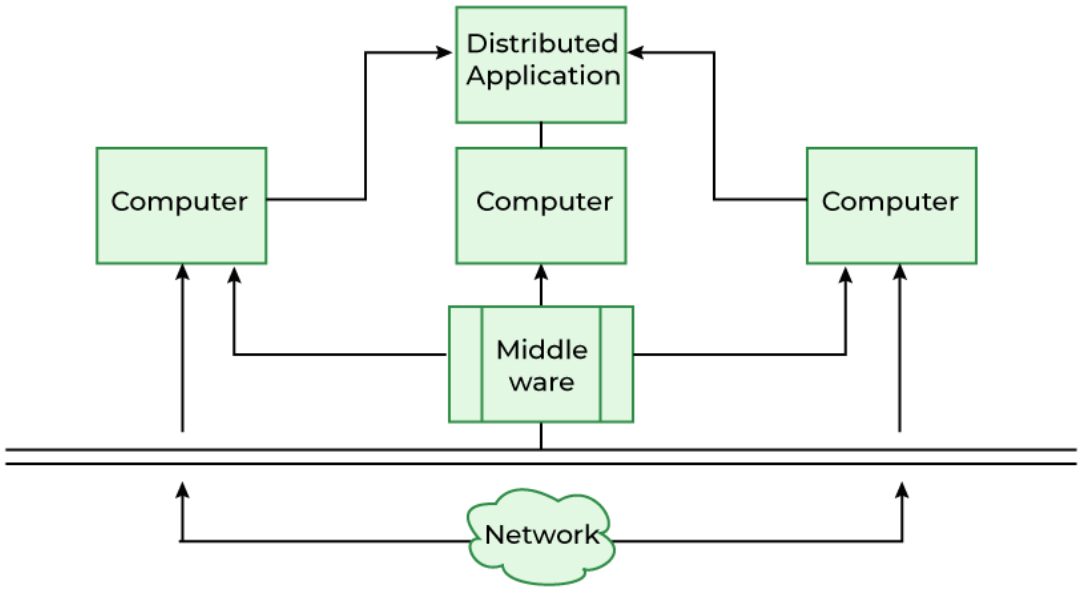


Image Credit: https://www.geeksforgeeks.org/operating-systems/types-of-distributed-system/

© 2026 by Innovation In Software Corporation

Distributed Systems

- Brittle component coupling
  - If service A and service B are tightly coupled through a shared, custom protocol
  - Then neither can change without coordinating with the other
  - In a system of any scale, this makes evolution of services nearly impossible.

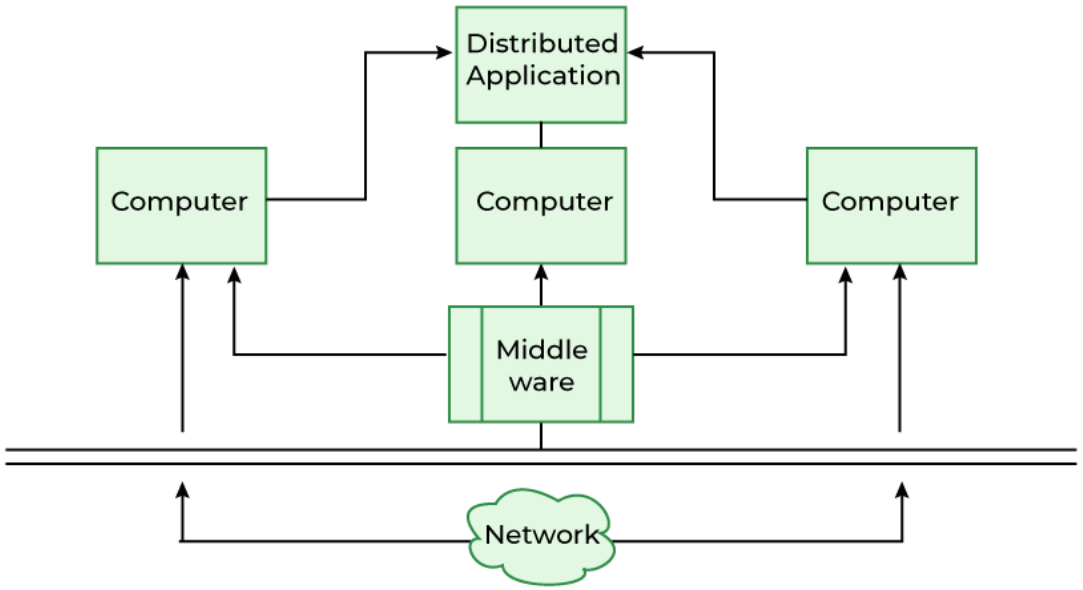
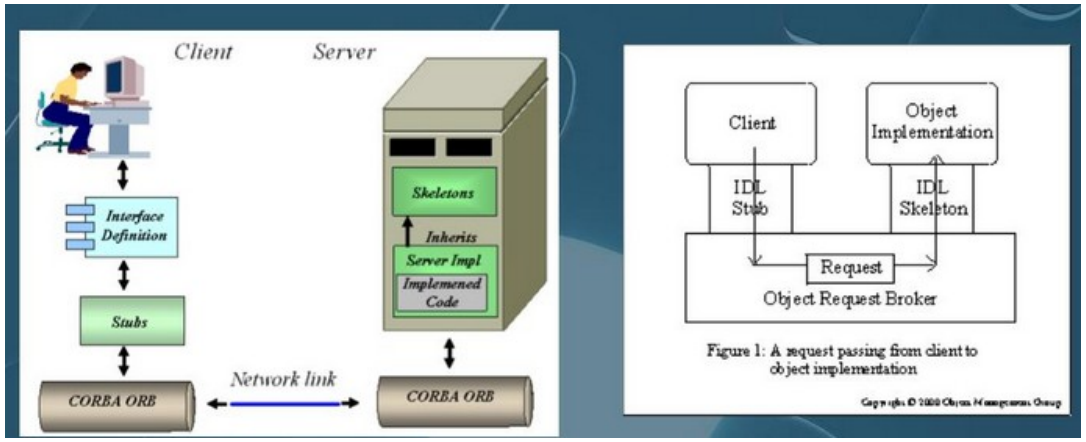


Image Credit: https://www.geeksforgeeks.org/operating-systems/types-of-distributed-system/

© 2026 by Innovation In Software Corporation

Partial Solutions

- CORBA and RPC
  - Solved discovery so components could find and call each other
  - But they required tight coupling to generated stubs
  - Every change to an interface broke every client contract with that interface
  - Infrastructure remained blind
- Custom HTTP APIs
  - Removed the stub coupling
  - But every team invented their own conventions, so there was no shared vocabulary
    - A client consuming ten APIs had to learn ten different HTTP dialects

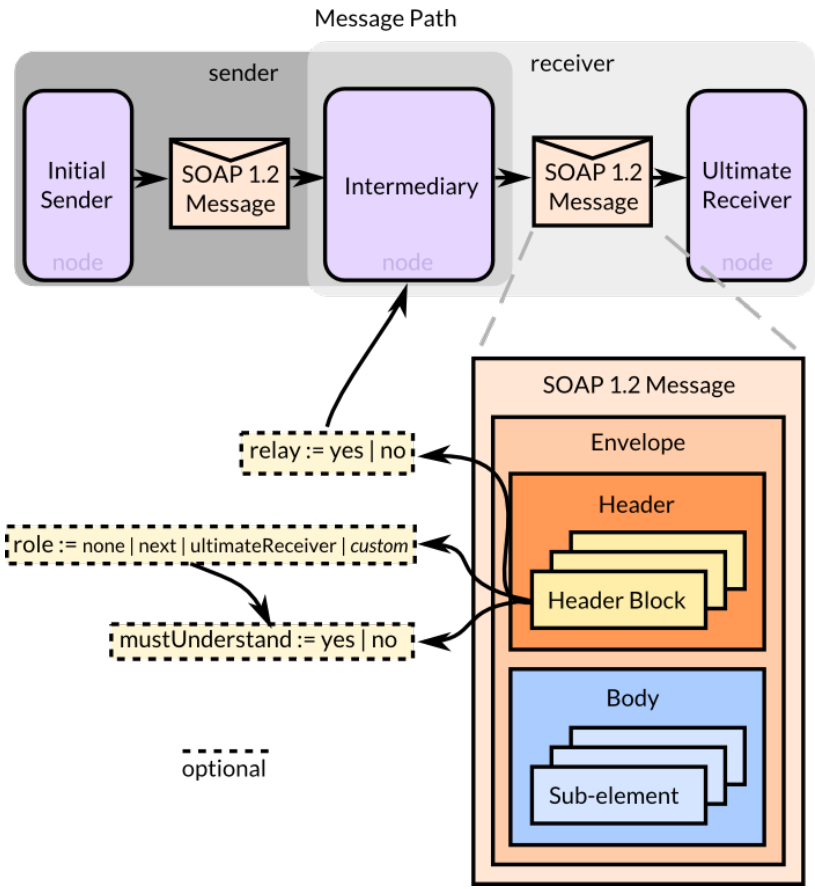


© 2026 by Innovation In Software Corporation

Image Credit: https://www.slideserve.com/abel-black/programming-heterogeneous-systems-with-corba-powerpoint-ppt-presentation

Partial Solutions

- SOAP
  - Most commonly implemented for Service Oriented Architecture
- What SOAP got right
  - Provided a universal contract through XML messaging
  - Provided discovery through WSDL
    - Web Services Description Language
    - Formally described every operation an exposed service, every input it accepted, and every output it returned
    - Provided a standard error vocabulary through fault codes



© 2026 by Innovation In Software Corporation

Image Credit: https://thecustomizewindows.com/2014/03/simple-object-access-protocol-soap/

Partial Solutions

- What SOAP got wrong
  - SOAP solved the discovery problem by focusing on the services contract
    - As a result, the contract was rigid, verbose, and tightly coupled to the implementation behind it
  - The envelope overhead was enormous
    - Every SOAP message was wrapped in an XML envelope containing headers and a body.
    - A request to retrieve a single record might carry ten times more XML infrastructure than actual data
    - Parsing that envelope on every request was expensive at the CPU level and expensive at the developer level

```xml
<definitions
    xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://cloud.dronemap.org
        /wsdl" name="MAVLinkControllerWSImplService">
    <types>
        <xsd:schema>
            <xsd:import namespace="http://cloud.dronemap.org/wsdl" schemaLocation="http
                ://192.168.1.13:3030/MAVLinkControllerService?xsd=1"/>
        </xsd:schema>
    </types>
    <message name="takeoff">
        <part name="parameters" element="tns:takeoff"/>
    </message>
    <message name="takeoffResponse">
        <part name="parameters" element="tns:takeoffResponse"/>
    </message>
    <portType name="MAVLink Action Web Service">
        <operation name="takeoff">
            <input wsam:Action="http://cloud.dronemap.org/wsdl/MAVLink Action Web Service
                /takeoffRequest" message="tns:takeoff"/>
            <output wsam:Action="http://cloud.dronemap.org/wsdl/MAVLink Action Web Service
                /takeoffResponse" message="tns:takeoffResponse"/>
        </operation>
    </portType>
    <binding name="MAVLinkControllerWSImplPortBinding" type="tns:MAVLink Action Web
        Service">
        <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
        <operation name="takeoff">
            <soap:operation soapAction=""/>
            <input>
                <soap:body use="literal"/>
            </input>
            <output>
                <soap:body use="literal"/>
            </output>
        </operation>
    </binding>
    <service name="MAVLinkControllerWSImplService">
        <port name="MAVLinkControllerWSImplPort" binding="tns
            :MAVLinkControllerWSImplPortBinding">
            <soap:address location="http://192.168.1.13:3030/MAVLinkControllerService"/>
        </port>
    </service>
</definitions>
```

© 2026 by Innovation In Software Corporation

Image Credit: https://www.researchgate.net/figure/Example-of-a-WSDL-document-of-SOAP-Web-Service-to-control-a-drone_fig1_330617455

Partial Solutions

- What SOAP got wrong
  - WSDL coupling was total
    - WSDL described operations, not resource
    - If the service changed in any way then the WSDL changed
    - Result was that every generated client stub broke
    - The contract locked producer and consumer together into a tight coupling
  - Infrastructure remained blind
    - HTTP used purely as a transport
    - Every message, regardless of it did, was a POST to a single endpoint
    - A load balancer could not make routing decisions based on the operation
    - Infrastructure that understood HTTP was useless because SOAP ignored HTTP semantics entirely.

```xml
<definitions
    xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://cloud.dronemap.org
        /wsdl" name="MAVLinkControllerWSImplService">
    <types>
        <xsd:schema>
            <xsd:import namespace="http://cloud.dronemap.org/wsdl" schemaLocation="http
                ://192.168.1.13:3030/MAVLinkControllerService?xsd=1"/>
        </xsd:schema>
    </types>
    <message name="takeoff">
        <part name="parameters" element="tns:takeoff"/>
    </message>
    <message name="takeoffResponse">
        <part name="parameters" element="tns:takeoffResponse"/>
    </message>
    <portType name="MAVLink Action Web Service">
        <operation name="takeoff">
            <input wsam:Action="http://cloud.dronemap.org/wsdl/MAVLink Action Web Service
                /takeoffRequest" message="tns:takeoff"/>
            <output wsam:Action="http://cloud.dronemap.org/wsdl/MAVLink Action Web Service
                /takeoffResponse" message="tns:takeoffResponse"/>
        </operation>
    </portType>
    <binding name="MAVLinkControllerWSImplPortBinding" type="tns:MAVLink Action Web
        Service">
        <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
        <operation name="takeoff">
            <soap:operation soapAction=""/>
            <input>
                <soap:body use="literal"/>
            </input>
            <output>
                <soap:body use="literal"/>
            </output>
        </operation>
    </binding>
    <service name="MAVLinkControllerWSImplService">
        <port name="MAVLinkControllerWSImplPort" binding="tns
            :MAVLinkControllerWSImplPortBinding">
            <soap:address location="http://192.168.1.13:3030/MAVLinkControllerService"/>
        </port>
    </service>
</definitions>
```

© 2026 by Innovation In Software Corporation

Image Credit: https://www.researchgate.net/figure/Example-of-a-WSDL-document-of-SOAP-Web-Service-to-control-a-drone_fig1_330617455

Partial Solutions

- What SOAP got wrong
  - Tooling dependency was absolute
    - Working with SOAP without a code-generation tool was impractical
    - The protocol was not human-readable in any useful sense
    - Debugging required specialized tools
    - Testing required specialized clients
    - The learning curve was steep and the operational overhead was high.

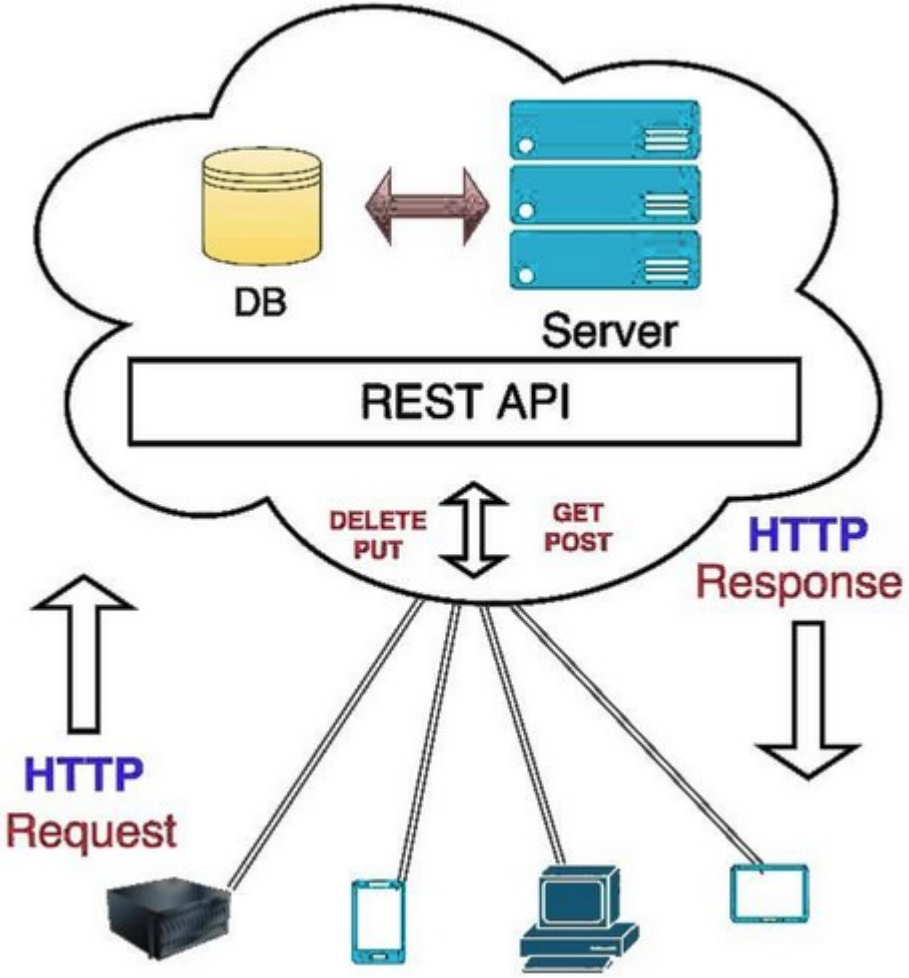
```xml
<definitions
    xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://cloud.dronemap.org
        /wsdl" name="MAVLinkControllerWSImplService">
    <types>
        <xsd:schema>
            <xsd:import namespace="http://cloud.dronemap.org/wsdl" schemaLocation="http
                ://192.168.1.13:3030/MAVLinkControllerService?xsd=1"/>
        </xsd:schema>
    </types>
    <message name="takeoff">
        <part name="parameters" element="tns:takeoff"/>
    </message>
    <message name="takeoffResponse">
        <part name="parameters" element="tns:takeoffResponse"/>
    </message>
    <portType name="MAVLink Action Web Service">
        <operation name="takeoff">
            <input wsam:Action="http://cloud.dronemap.org/wsdl/MAVLink Action Web Service
                /takeoffRequest" message="tns:takeoff"/>
            <output wsam:Action="http://cloud.dronemap.org/wsdl/MAVLink Action Web Service
                /takeoffResponse" message="tns:takeoffResponse"/>
        </operation>
    </portType>
    <binding name="MAVLinkControllerWSImplPortBinding" type="tns:MAVLink Action Web
        Service">
        <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
        <operation name="takeoff">
            <soap:operation soapAction=""/>
            <input>
                <soap:body use="literal"/>
            </input>
            <output>
                <soap:body use="literal"/>
            </output>
        </operation>
    </binding>
    <service name="MAVLinkControllerWSImplService">
        <port name="MAVLinkControllerWSImplPort" binding="tns
            :MAVLinkControllerWSImplPortBinding">
            <soap:address location="http://192.168.1.13:3030/MAVLinkControllerService"/>
        </port>
    </service>
</definitions>
```

© 2026 by Innovation In Software Corporation

Image Credit: https://www.researchgate.net/figure/Example-of-a-WSDL-document-of-SOAP-Web-Service-to-control-a-drone_fig1_330617455

Uniform Interface

- Without a uniform interface
  - Every client needs customized knowledge of every server
  - Caching is impossible
    - No standard way to signal reuse
  - Intermediate layers (proxies, gateways) cannot interpret requests
  - Monitoring tools cannot classify errors without parsing response bodies
  - APIs accumulate unique conventions that new developers must learn from scratch



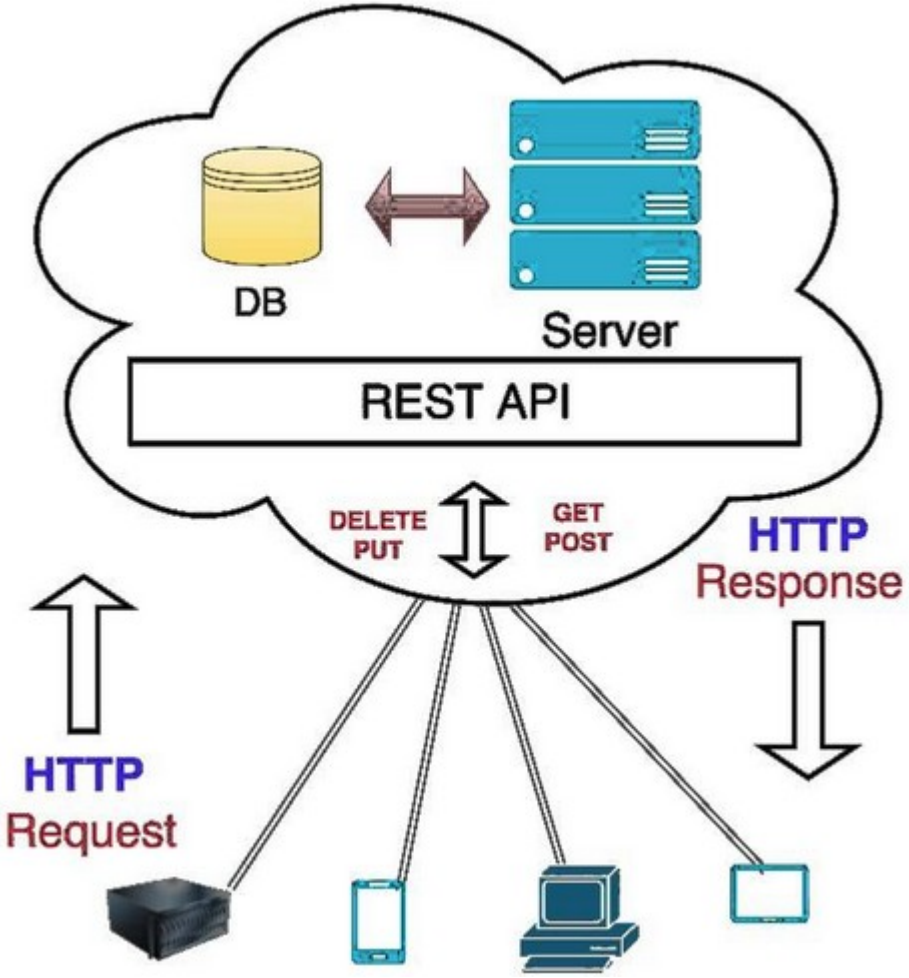
© 2026 by Innovation In Software Corporation

Resource-Oriented URL Design

© 2026 by Innovation In Software Corporation

Uniform Interface

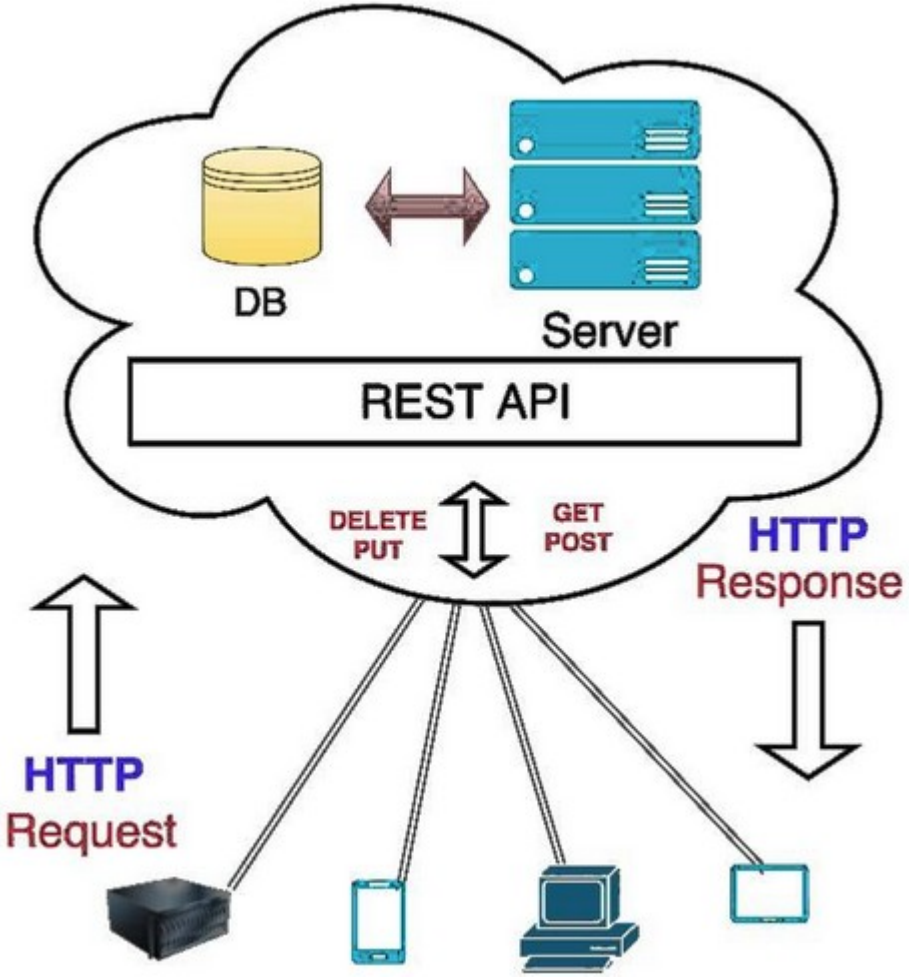
- REST: a uniform interface contract
  - URLs identify resources so that any client can understand the address
  - HTTP methods carry action semantics as a standardized vocabulary
  - Status codes signal outcomes which are readable by any infrastructure layer
  - GET is safe and cacheable so that proxies and CDNs can participate
  - Any HTTP-aware system can interact with any REST API without prior coordination



© 2026 by Innovation In Software Corporation

REST: Architectural Constraints

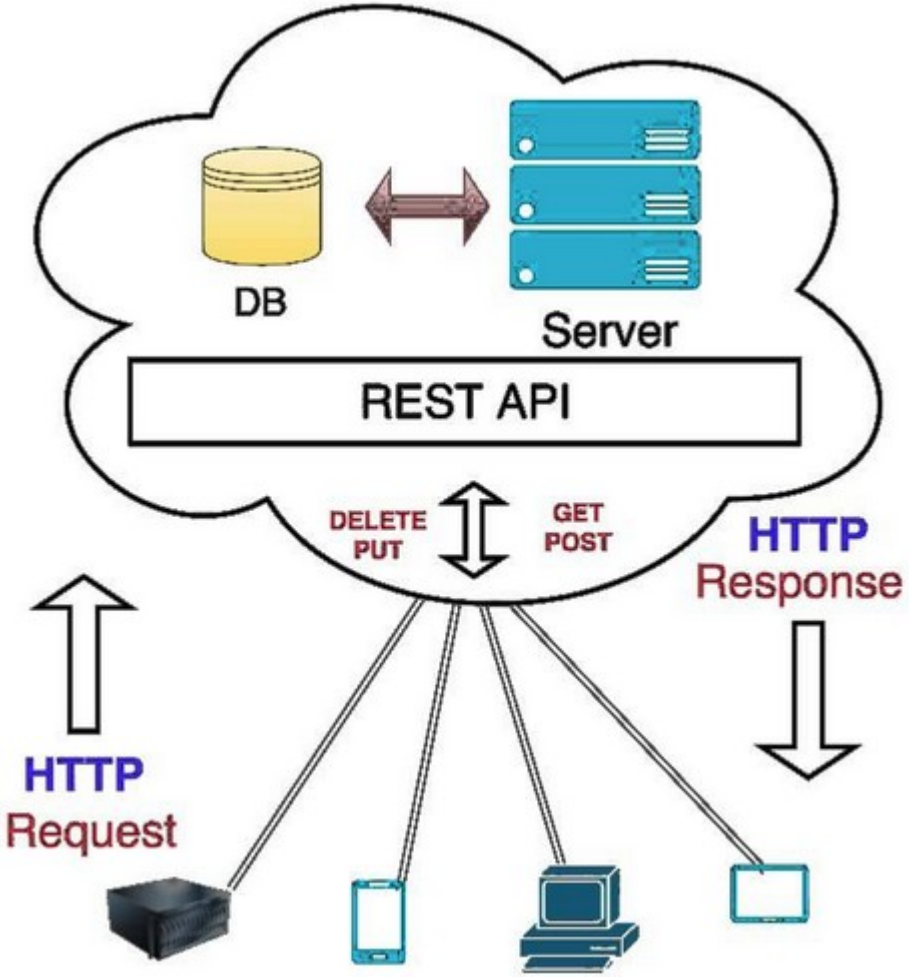
- Representational State Transfer
  - Roy Fielding, Doctoral dissertation, 2000
  - Not a protocol - no wire format
  - Not a specification - no RFC to implement
  - A set of constraints on distributed system architecture
  - Applied specifically to hypermedia systems
  - Designed by observing what had made it possible for the web to scale in the past



© 2026 by Innovation In Software Corporation

REST: Architectural Constraints

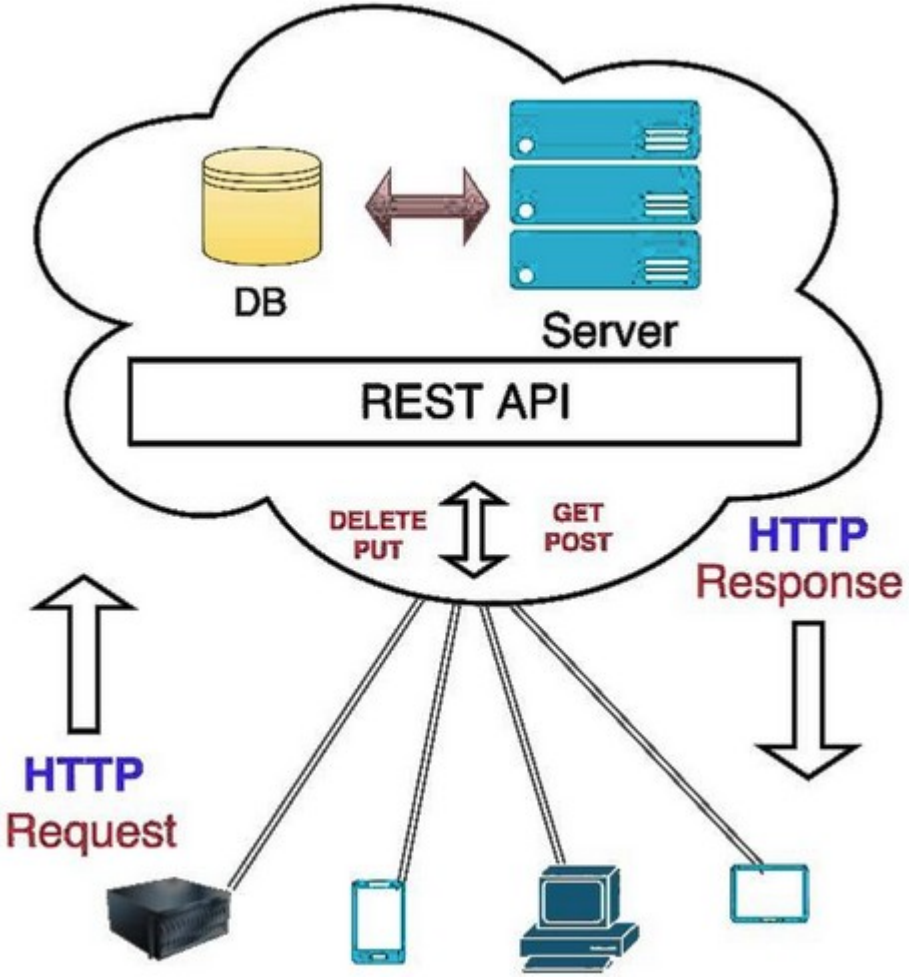
- Constraints on server/browser interactions
  - Browser has no prior knowledge of any specific server
  - Server has no prior knowledge of any specific browser
  - Both agree: URLs identify resources
  - Both agree: HTTP methods carry semantics
  - Both agree: status codes signal outcomes
  - Result: any client can talk to any server



© 2026 by Innovation In Software Corporation

Four Properties

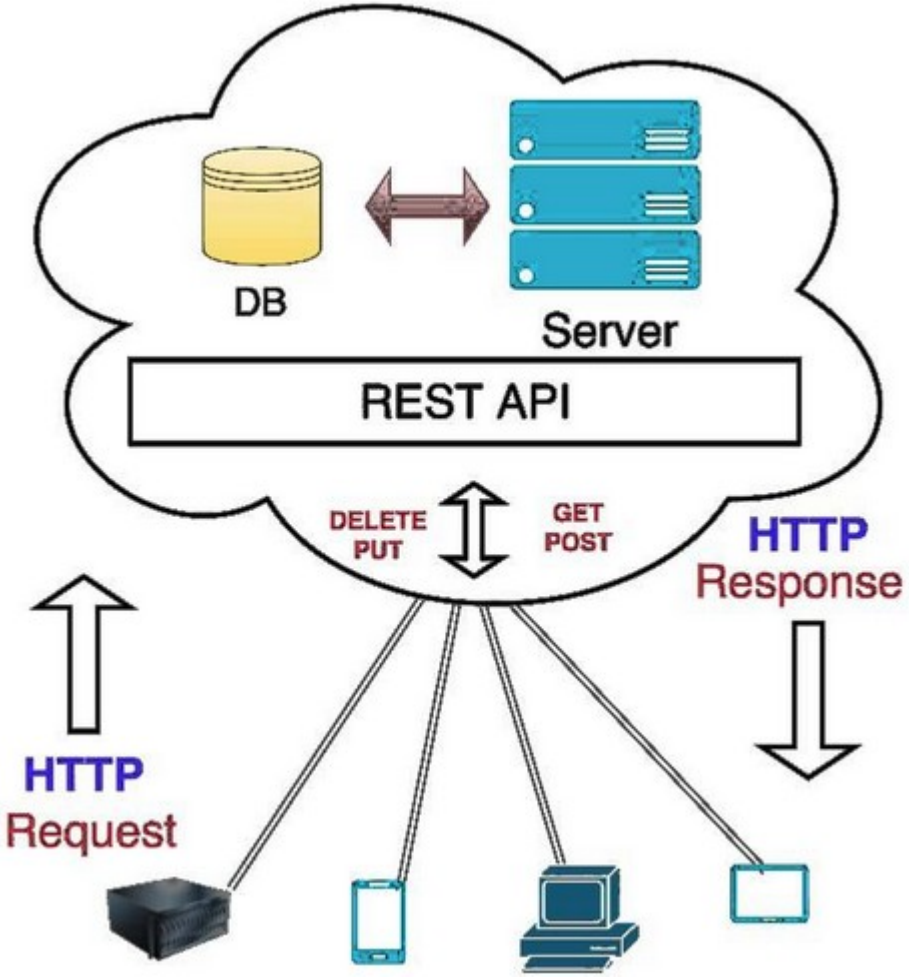
- Interoperability
  - Any client that understands HTTP can interact with any REST server
  - No custom protocol negotiation.
  - No client-specific adapters required
- Cacheability
  - GET is safe and idempotent.
  - CDNs, load balancers, and API gateways can serve cached responses without application-level knowledge



© 2026 by Innovation In Software Corporation

Four Properties

- Evolvability
  - Server implementations can change internally as long as the interface contract is preserved
  - The URL is the boundary between public API and private implementation
- Observability
  - Uniform status codes let monitoring, alerting, and log aggregators detect and classify errors without parsing response bodies



© 2026 by Innovation In Software Corporation

Six Architectural Constraints

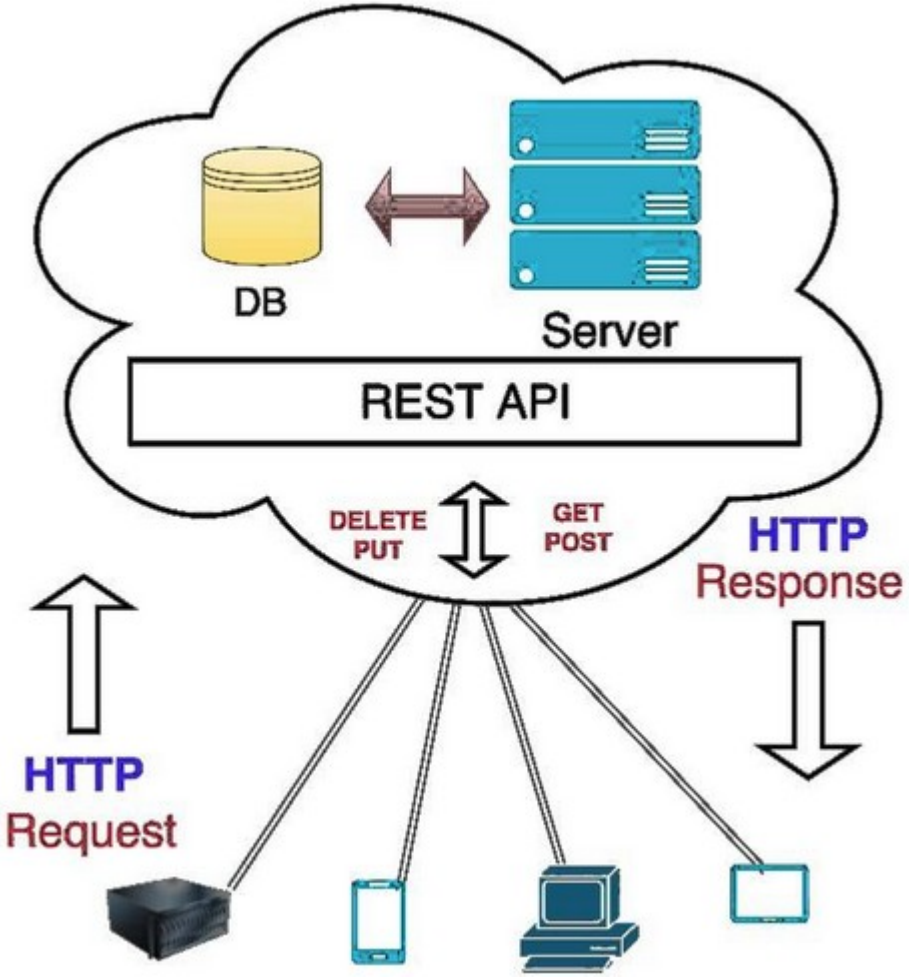
- Each constraint is a design decision with a traceable practical consequence

Constraint	Practical Consequence
Client-Server	Frontend and backend evolve independently. UI is separated from data storage.
Stateless	Every request carries all information needed. No session state on the server — tokens carry identity.
Cacheable	Responses declare cacheability. Infrastructure layers (CDN, gateway) serve responses without hitting the application.
Uniform Interface	URL design, method semantics, and status codes form the contract. The internet itself is proof this works at scale.
Layered System	Clients cannot tell if connected to the origin server or an intermediary. Load balancers and API gateways are transparent.
Code on Demand *(optional)*	Servers can transfer executable code to clients. Rarely relevant for internal API design.

© 2026 by Innovation In Software Corporation

The Stateless Constraint

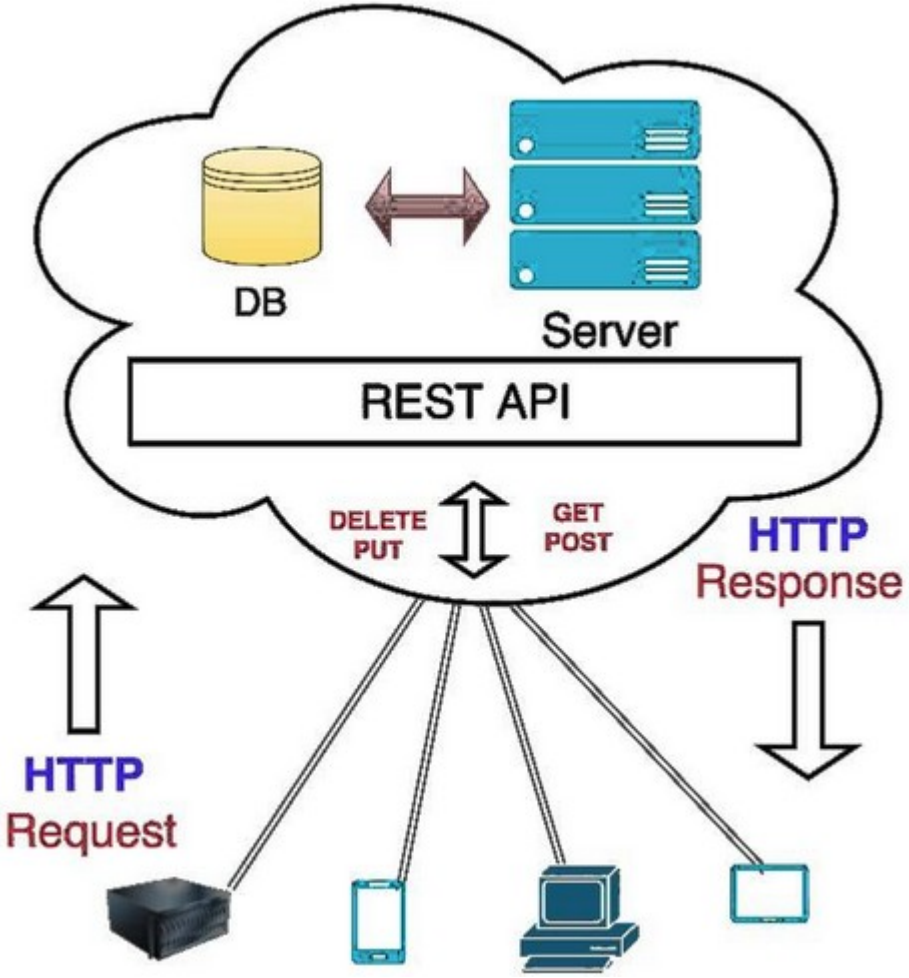
- Foundation of token-based auth
  - The server stores no session state between requests
  - Every request must contain all information required to process it
- Session-based auth (stateful)
  - Server stores session in memory or a shared cache
  - All requests must reach the same server, or session storage must be shared and replicated
  - Logout requires session invalidation across all instances
  - Horizontal scaling adds significant infrastructure complexity



© 2026 by Innovation In Software Corporation

The Stateless Constraint

- Token-based auth (stateless)
  - Server stores no session, credentials travel with each request
  - Any instance can handle any request which enables horizontal scaling
  - JWT (JSON web token) carries identity and claims, signed and verifiable offline
  - Revocation handled via token expiry or a token introspection endpoint
  - The OAuth2/JWT architecture is the direct application of this constraint



© 2026 by Innovation In Software Corporation

The Stateless Constraint

- Session-based auth creates server-side state
  - Must be replicated across instances, invalidated on logout, and managed at scale
- Token-based auth externalizes that state to the token itself which verifiable by any instance with access to the signing key
  - Scale
    - Any service instance handles any request
    - Pure horizontal scaling
    - No sticky sessions
  - Security
    - JWT claims travel with the request
    - No hidden server-side state that can be hijacked or forged

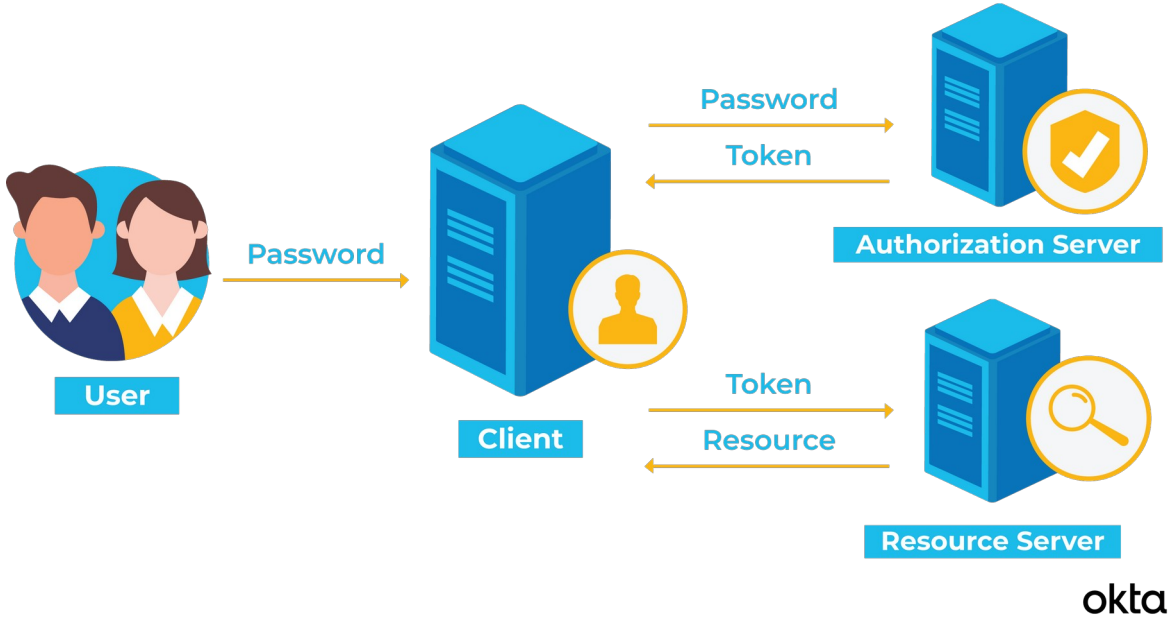


Image Credit:https://www.okta.com/identity-101/what-is-token-based-authentication/

© 2026 by Innovation In Software Corporation

The Uniform Interface

- Resource identification
  - Every resource is identified by a URI
  - The URI is the address, not the implementation, or the database table, or the class name
- Manipulation via representations
  - Clients interact with resources through representations (JSON, XML)
  - The server's internal model is hidden behind the representation

Constraint	Design Decision
Stateless	Bearer token on every request — no session cookies. JWT carries identity. No server-side session store.
Uniform Interface	Nouns in URLs. HTTP method carries the verb. Standard status codes on every response.
Cacheable	GET has no side effects. Cache-Control: no-store on authenticated responses. CDN-eligible public responses return ETag.
Layered System	API gateway, load balancer, and circuit breaker sit between client and service transparently.
Client-Server	React SPA and Spring Boot REST API evolve on separate release cycles. No server-side rendered HTML.

© 2026 by Innovation In Software Corporation

The Uniform Interface

- Self-descriptive messages
  - Each message includes enough information to describe how to process it
    - Content-Type, status code, standard headers
- HATEOAS
  - Hypermedia as the Engine of Application State
  - Responses include links to valid next actions
  - Clients follow links returned by the server rather than constructing URLs themselves

Constraint	Design Decision
Stateless	Bearer token on every request — no session cookies. JWT carries identity. No server-side session store.
Uniform Interface	Nouns in URLs. HTTP method carries the verb. Standard status codes on every response.
Cacheable	GET has no side effects. Cache-Control: no-store on authenticated responses. CDN-eligible public responses return ETag.
Layered System	API gateway, load balancer, and circuit breaker sit between client and service transparently.
Client-Server	React SPA and Spring Boot REST API evolve on separate release cycles. No server-side rendered HTML.

© 2026 by Innovation In Software Corporation

Summary

- REST is a design
  - Six architectural constraints, each with a traceable practical consequence for an API design
  - Uniform interface makes scale possible
    - URLs identify resources
    - HTTP methods carry semantics
    - Status codes signal outcomes
    - Any client can talk to any server
  - Stateless is an architectural decision
    - No server session state because tokens travel with each request
  - Interoperability, cacheability, evolvability, and observability are consequences of applying the constraints correctly

Constraint	Practical Consequence
Client-Server	Frontend and backend evolve independently. UI is separated from data storage.
Stateless	Every request carries all information needed. No session state on the server — tokens carry identity.
Cacheable	Responses declare cacheability. Infrastructure layers (CDN, gateway) serve responses without hitting the application.
Uniform Interface	URL design, method semantics, and status codes form the contract. The internet itself is proof this works at scale.
Layered System	Clients cannot tell if connected to the origin server or an intermediary. Load balancers and API gateways are transparent.
Code on Demand *(optional)*	Servers can transfer executable code to clients. Rarely relevant for internal API design.

© 2026 by Innovation In Software Corporation

Resource-Oriented URL Design

- A URL identifies a thing
  - It is not an action
  - The thing is some resource representing a domain object like a transaction
  - The CRUD action on the resource is expressed by the HTTP method
- When URLs carry verbs, the uniform interface breaks down
  - Infrastructure cannot interpret the operation
  - Clients must learn each API's private verb vocabulary
  - Caching becomes impossible because the URL no longer reliably identifies a resource because it identifies an operation

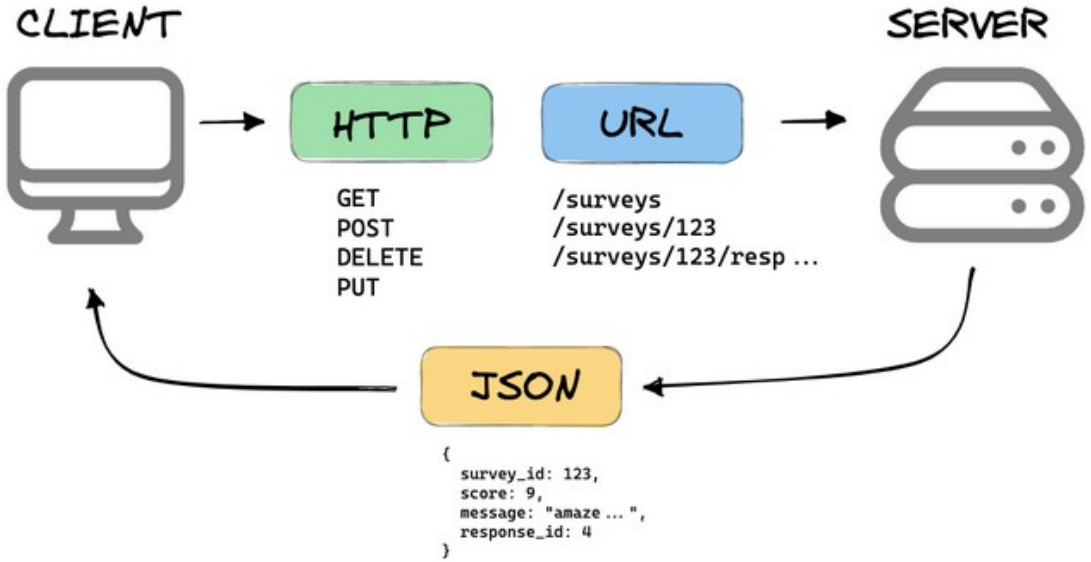


Image Credit: https://mannhowie.com/rest-api

© 2026 by Innovation In Software Corporation

Resource-Oriented URL Design

- The mental model shift
  - A URL is an address, not a command
  - You do not POST to /createEmployee, you POST to /employees
  - Creation intent is stated by the POST method, not the URL
  - The URL /employees refers to the same thing regardless of which method is applied to it
- The status code and headers returned
  - Are part of the API contract, not part of implementation detail
  - Clients, monitoring systems, and retry logic all depend on them being correct and consistent

Method	Path	Intent	Response Contract
GET	/employees	Return paginated list	200 OK with body. Safe, idempotent, cacheable.
GET	/employees/{id}	Return single employee	200 OK with body, or 404 if not found
POST	/employees	Create new employee	201 Created with Location: /employees/{newId} header
PUT	/employees/{id}	Replace entire resource	200 OK with body, or 204 No Content. Idempotent.
PATCH	/employees/{id}	Partial update	200 OK with updated representation
DELETE	/employees/{id}	Remove employee	204 No Content. Idempotent.
GET	/employees/{id}/roles	Roles belonging to employee	200 OK. Sub-resource collection.

© 2026 by Innovation In Software Corporation

URL Hierarchies and Sub-Resources

- A sub-resource URL like

  /employees/{id}/roles

  - Expresses an ownership relationship
    - Roles belong to an employee
    - Their lifecycle is bound to the parent
    - A role without an employee has no meaning in the domain

When to introduce a top-level resource instead of a sub-resource

Situation	Approach
The child has meaning independent of the parent	Top-level resource: `/roles/{id}`
The hierarchy is deeper than two levels	Break out the deepest level as a top-level resource
Multiple parent types can own the child	Top-level resource with a parent reference as a query parameter
The child is queried frequently without the parent	Top-level resource to avoid forcing clients to know the parent ID

© 2026 by Innovation In Software Corporation

URL Hierarchies and Sub-Resources

- A sub-resource URL
  - The rule for sub-resources
    - Use a hierarchy when the child resource's identity is *only* meaningful in the context of the parent
    - The hierarchy should reflect domain ownership, not database join relationships
  - The depth rule
    - Go at most two levels deep

      /employees/{id}/roles/{roleId}/permissions/{permissionId}
    - More than two levels here is a hint that permissions should be a top-level resource with their own identity

When to introduce a top-level resource instead of a sub-resource

Situation	Approach
The child has meaning independent of the parent	Top-level resource: `/roles/{id}`
The hierarchy is deeper than two levels	Break out the deepest level as a top-level resource
Multiple parent types can own the child	Top-level resource with a parent reference as a query parameter
The child is queried frequently without the parent	Top-level resource to avoid forcing clients to know the parent ID

© 2026 by Innovation In Software Corporation

Query Parameters

- Query parameters
  - Refine which representation of a resource collection is returned
    - They are not part of the resource identity
    - They are instructions to the server about how to shape the response
    - A URL with different query parameters still refers to the same underlying resource
  - The names sort, page, size, filter, cursor are industry conventions
    - They are widely understood and predictable
    - They are not mandated by REST, HTTP, or any RFC that governs REST APIs.
    - Different APIs use different names, REST doesn't care what they are called as long as is a well formed query string

Filtering

```
GET /employees?department=engineering&status=active
GET /employees?hiredAfter=2022-01-01&location=london
```

Pagination

```
GET /employees?page=2&size=20
GET /employees?cursor=eyJpZCI6MTAwfQ==&limit=20
```

Sorting

```
GET /employees?sort=lastName,asc&sort=startDate,desc
```

Field projection

```
GET /employees?fields=id,firstName,lastName,email
```

© 2026 by Innovation In Software Corporation

Pagination

- Most common oversight in initial API implementations
  - Often has the most severe production consequences
  - Any endpoint that returns a collection must be paginated from the first implementation
  - In development
    - The test dataset has 20 records
    - The query returns instantly
    - The response payload is small
  - In production
    - The same endpoint may return 200,000 records
    - A single unbounded query can hold a database connection open for minutes
    - An unbounded response payload can exhaust application server memory
    - A single client request can trigger a cascading failure across the service

Filtering

```
GET /employees?department=engineering&status=active
GET /employees?hiredAfter=2022-01-01&location=london
```

Pagination

```
GET /employees?page=2&size=20
GET /employees?cursor=eyJpZCI6MTAwfQ==&limit=20
```

Sorting

```
GET /employees?sort=lastName,asc&sort=startDate,desc
```

Field projection

```
GET /employees?fields=id,firstName,lastName,email
```

© 2026 by Innovation In Software Corporation

Pagination

- The contract to establish from day one
  - Default page size is explicit, typically 20
  - Maximum page size is bounded, reject requests above a defined ceiling, typically 100
  - Total count and pagination metadata are always returned in the response envelope
  - Navigation links (first, previous, next, last) are included so clients do not construct URLs manually
  - A paginated collection response must return metadata alongside the data
    - Otherwise clients cannot determine whether more pages exist, how to request the next page, or how many total resources are available

```
{
  "data": [
    { "id": 1, "firstName": "Alice", "lastName": "Chen" },
    { "id": 2, "firstName": "Bob",   "lastName": "Diaz"  }
  ],
  "pagination": {
    "page": 2,
    "size": 20,
    "totalElements": 347,
    "totalPages": 18,
    "first": false,
    "last": false
  },
  "_links": {
    "self":  { "href": "/api/v1/employees?page=2&size=20" },
    "first": { "href": "/api/v1/employees?page=0&size=20" },
    "prev":  { "href": "/api/v1/employees?page=1&size=20" },
    "next":  { "href": "/api/v1/employees?page=3&size=20" },
    "last":  { "href": "/api/v1/employees?page=17&size=20" }
  }
}
```

© 2026 by Innovation In Software Corporation

URL Design Anti-Patterns

Anti-Pattern	Example	Problem	Correct Form
Verb in URL	POST /createEmployee	Breaks uniform interface. Infrastructure cannot interpret the operation from the URL alone.	POST /employees
RPC-style with query verb	GET /getEmployeeById?id=42	Verb in the URL, resource not identified by path. Not cacheable by path.	GET /employees/42
Method tunnelling	POST /employees/delete	Uses POST to perform a DELETE. Loses idempotency guarantees. Infrastructure cannot distinguish from a create.	DELETE /employees/{id}
Hierarchy too deep	GET /employees/42/roles/5/permissions/3	Signals a missing top-level resource. Forces clients to know the full ancestor chain.	GET /permissions/3
Action endpoint for simple filter	POST /employees/search	Acceptable for complex search payloads. Unnecessary for simple filtering.	GET /employees?department=engineering&status=active
Plural inconsistency	GET /employee vs GET /employees/42	Forces clients to remember per-resource pluralisation rules.	Use plural consistently: /employees and /employees/{id}

© 2026 by Innovation In Software Corporation

The Engineering Principles

Convention	Constraint It Applies	Consequence of Violation
Nouns in URLs, HTTP method carries the verb	Uniform interface — resource identification	Infrastructure cannot interpret the operation. Caching breaks. Monitoring misclassifies traffic.
Hierarchy reflects domain ownership, not database schema	Client-server separation — implementation hidden behind interface	Deep hierarchies leak the data model to consumers. Schema changes break URL structure.
Query parameters for filtering and sorting, not path segments	Uniform interface — resource identified by path	Path-encoded filters make URLs uncacheable and non-composable. Resources lose stable identity.
Pagination mandatory on all collection endpoints	Stateless constraint — each request is self-contained and bounded	Unbounded collection responses exhaust memory, hold database connections, and cause cascading failures under load.
Consistent pluralisation across all resources	Uniform interface — predictability of the contract	Inconsistency forces clients to memorise per-resource rules. Reduces API discoverability.

© 2026 by Innovation In Software Corporation

Summary

- A URL is an address, not a command
  - URLs identify resources
  - HTTP methods express what you want to do to them
  - When this separation is maintained, the entire HTTP infrastructure
    - Caches, proxies, gateways, monitoring, can intelligently support the applications
- Sub-resources express domain ownership, not database joins
  - Use hierarchy when the child's identity only makes sense in the context of the parent
  - Go at most two levels deep
- Query parameters refine a resource representation
  - They do not define a new resource
  - Filtering, sorting, and pagination are instructions to the server, not part of the resource address
- Pagination is a design decision, not an optimization
  - It must be present from the first implementation
  - It cannot be retrofitted without a breaking change

© 2026 by Innovation In Software Corporation

Summary

- The anti-patterns all have the same root cause
  - Verbs in URLs, method tunnelling, and unbounded collections
  - All occur when the HTTP uniform interface is ignored in favour of a local convention
  - The uniform interface exists precisely to prevent this divergence

© 2026 by Innovation In Software Corporation

HTTP Status Code Discipline

© 2026 by Innovation In Software Corporation

HTTP Status Code Discipline

- When a client calls an API
  - Three separate audiences need to understand what happened
    - The client application that made the call
    - The infrastructure sitting between client and server
    - The humans and tools monitoring the system in production.
  - Status codes serve all three simultaneously
  - There should be no need for any of them to parse a response body

© 2026 by Innovation In Software Corporation

HTTP Status Code Discipline

- The client application
  - Uses the status code to decide what to do next
    - A 200 means process the response
    - A 404 means the resource does not exist so stop retrying
    - A 401 means authenticate first
    - A 503 means the server is unavailable so retry with backoff
  - None of these decisions require reading the body
  - The status code is sufficient

© 2026 by Innovation In Software Corporation

HTTP Status Code Discipline

- Infrastructure
  - Load balancers, API gateways, circuit breakers, CDNs, etc.
  - Uses status codes to make routing and caching decisions
    - A CDN knows a 200 response to a GET may be cacheable
    - A circuit breaker counts 5xx responses to decide when to open
    - A load balancer stops sending traffic to an instance returning 503
  - None of this is possible if the application returns a code 200 for everything and encodes the real outcome in the body

© 2026 by Innovation In Software Corporation

HTTP Status Code Discipline

- Monitoring and alerting
  - Classifies traffic by status code
  - A spike in 4xx tells operators that clients are sending bad requests
    - Likely a client bug or a breaking API change
  - A spike in 5xx tells operators the server is failing
    - Likely a deployment issue or a dependency outage
  - A system that returns 200 for errors hides both signals and makes production incidents significantly harder to diagnose
    - Inventing a parallel outcome signalling mechanism inside the response body is the one of most costly REST design mistake in production systems

© 2026 by Innovation In Software Corporation

The Five Status Code Classes

- Status codes are grouped into classes
  - The first digit of every status code identifies the class
  - Every HTTP client, infrastructure component, and monitoring tool understands these five classes without any application-specific configuration
- Important distinction: 4xx versus 5xx
  - A 4xx means the client sent a bad request and the server behaved correctly.
  - A 5xx means the server failed to handle a valid request
    - Confusing these two classes produces wrong metrics, wrong alerting, and wrong client behavior
    - Returning 500 for a validation failure is not a minor inaccuracy
    - It misdirects operators to look for server problems when the real problem is the client

Class	Range	Meaning
1xx Informational	100–199	Request received, processing continues. Rarely relevant for REST APIs.
2xx Success	200–299	The request was received, understood, and accepted. The specific code communicates the nature of the success.
3xx Redirection	300–399	Further action is needed to complete the request. Used for redirects and resource moves.
4xx Client Error	400–499	The request contains bad syntax or cannot be fulfilled. The fault is with the client. Retrying the same request without changing it will not help.
5xx Server Error	500–599	The server failed to fulfil a valid request. The fault is with the server. The client may retry.

© 2026 by Innovation In Software Corporation

2xx Success

- The 201 contract is frequently violated
  - Returning 200 instead of 201 removes the Location header
    - This is the mechanism by which clients discover the URI of the newly created resource without making a second request.
    - POST endpoints that create resources return 201 with a Location header.
  - Returning 204 with a body is a protocol error
    - If you need to return the updated resource after a PUT or PATCH, return 200 with the body
    - If you have nothing to return, return 204 with no body
    - Choose one and be consistent across all endpoints

Code	Name	When to Use
200	OK	Successful GET, PUT, or PATCH where a response body is returned. The most general success code. Never return 200 with an error body.
201	Created	Successful POST that creates a new resource. Must include a `Location` response header pointing to the URI of the new resource: `Location: /api/v1/employees/42`. The body may contain the created resource representation.
204	No Content	Successful operation with no response body. Used for DELETE, or for PUT and PATCH when the updated representation is not returned. The response body must be empty — literally nothing.

© 2026 by Innovation In Software Corporation

4xx Client Error

- The table shows the most relevant client error codes that should be returned
  - Returning the correct code is important
  - It helps the client determine how to handle the error
    - The client should not have to parse the body of the response in order to determine how to handle the error
  - Monitoring systems can use surges in occurrences of certain errors, like 401/403, that might indicate an attack or other security problem

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

5xx Server Error

- Security dimension
  - A 500 response must never contain
    - A stack trace, internal class names, database error messages, SQL fragments, or any other implementation detail
    - Each of these can be reconnaissance information for an attacker
  - The full exception is logged internally, associated with a trace ID
  - The client receives the trace ID and nothing else about the internal failure

Code	Name	When to Use
500	Internal Server Error	An unexpected fault occurred in the server. The request may have been valid. The server failed to handle it. Log the full exception internally. Return a sanitised error payload with a trace ID to the client. Never return a stack trace.
502	Bad Gateway	The server, acting as a gateway or proxy, received an invalid response from an upstream service. The upstream is the source of the failure.
503	Service Unavailable	The server is temporarily unable to handle requests. A circuit breaker is open, a dependency is down, or the server is under extreme load. Include a `Retry-After` header if the duration is known.
504	Gateway Timeout	The server, acting as a gateway, did not receive a timely response from an upstream service. Distinct from 503 — the upstream is reachable but slow.

© 2026 by Innovation In Software Corporation

401 vs 403 Distinction

- 401 Unauthorized
  - *"I do not know who you are"*
  - Authentication is required but has not been provided, or the provided credentials are invalid
  - The implicit message to the client
    - If you provide valid credentials, you may get access
    - The WWW-Authenticate header must be included, indicating the authentication scheme
  - Typical causes of the error
    - Missing Authorization header, expired JWT, invalid JWT signature, token issued by an untrusted issuer

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

401 vs 403 Distinction

- 403 Forbidden
  - *"I know exactly who you are, and you cannot do this"*
  - The client is authenticated
    - The token is valid
    - The decision is authorization, not authentication
  - The implicit message to the client
    - Providing different credentials will not help because you lack the required permission
    - No WWW-Authenticate header
  - Typical causes of the error
    - Insufficient scope, missing role, attempting to access another user's resource

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

404 vs 403 Distinction

- Returning 403
  - Confirms to the caller that the resource exists
  - But when knowing of the existence of a resource is itself sensitive
    - It is preferable to return 404
    - Makes the response indistinguishable from a genuinely absent resource
    - This prevents resource enumeration attacks where an attacker probes IDs to discover which resources exist
  - Best practice
    - When the existence of a resource is sensitive, unauthenticated or unprivileged access returns 404
    - When the resource's existence is not sensitive and the caller is authenticated, return 403 so the client understands that authentication is not the issue

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

400 vs 422 Distinction

- Both codes indicate a client error involving the request payload
  - The distinction is the level at which the request fails
- 400 Bad request
  - The request cannot be understood
  - The server cannot parse or structurally process the request as sent, for example
    - Malformed JSON that cannot be deserialized
    - Missing required fields that the deserializer needs to construct the request object
    - Field values of the wrong type, for example, a string where an integer is required
    - Failed Bean Validation constraints like @NotBlank, @Min, @Email, @Size
  - In Spring Boot, MethodArgumentNotValidException and HttpMessageNotReadableException both map to 400

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

400 vs 422 Distinction

- 422 Unprocessable entity
  - The request is understood but cannot be executed
  - The request is syntactically valid and passes field-level validation
  - The server understands it completely
  - But the operation violates a business rule given the current state of the domain
  - Examples
    - Attempting to approve a leave request that has already been approved
    - Attempting to transfer an amount that exceeds the available balance
    - Attempting to assign a role to a user who already holds that role
    - Attempting to schedule a meeting in a room that is already booked for that slot

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

400 vs 422 Distinction

- The distinction matters because the appropriate client response is different
  - A 400 tells the client to fix the request before retrying
  - A 422 tells the client that the request itself is fine
    - But the problem is the current state of the system
    - The client may need to query current state before deciding whether or how to retry

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

The 200-With-Error Anti-Pattern

- One of most damaging REST design mistake
  - Occurs when HTTP is seen as only transport layer rather than as the uniform interface
  - Every client must parse the response body before it can determine whether the call succeeded
  - Infrastructure becomes blind
    - A CDN may cache this 200 response
    - A circuit breaker will not count it as a failure
    - A monitoring dashboard will show 100% success rate while the application is returning errors on every call

Code	Name	When to Use
200	OK	Successful GET, PUT, or PATCH where a response body is returned. The most general success code. Never return 200 with an error body.
201	Created	Successful POST that creates a new resource. Must include a `Location` response header pointing to the URI of the new resource: `Location: /api/v1/employees/42`. The body may contain the created resource representation.
204	No Content	Successful operation with no response body. Used for DELETE, or for PUT and PATCH when the updated representation is not returned. The response body must be empty — literally nothing.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "success": false,
  "errorCode": "EMPLOYEE_NOT_FOUND",
  "message": "Employee with id 42 does not exist"
}
```

© 2026 by Innovation In Software Corporation

The 200-With-Error Anti-Pattern

- Problems
  - Retry logic breaks
    - A client that retries on 5xx will not retry
    - A client will may not implement the correct retry logic

The correct form:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "status": 404,
  "error": "EMPLOYEE_NOT_FOUND",
  "message": "Employee with id 42 does not exist",
  "path": "/api/v1/employees/42",
  "timestamp": "2024-11-15T14:32:00Z",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

© 2026 by Innovation In Software Corporation

Status Codes in Spring Boot

- Status codes in Spring Boot are set in three ways
  - Common mistake is hardcoding status codes in controller methods
- @ResponseStatus on exception classes
  - Preferred for domain exceptions
  - Thrown anywhere in the service layer
  - In the example
    - Caught by Spring and mapped to 404 automatically

```java
@ResponseStatus(HttpStatus.NOT_FOUND)
public class EntityNotFoundException extends RuntimeException {
    public EntityNotFoundException(String message) {
        super(message);
    }
}
```

© 2026 by Innovation In Software Corporation

Status Codes in Spring Boot

- @ControllerAdvice with @ExceptionHandler
  - Preferred for centralized mapping
  - All exception-to-status-code mappings live in one place
  - Controllers contain no error handling logic

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError handleValidation(MethodArgumentNotValidException ex) { ... }

    @ExceptionHandler(EntityNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiError handleNotFound(EntityNotFoundException ex) { ... }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ApiError handleUnexpected(Exception ex) { ... }
}
```

© 2026 by Innovation In Software Corporation

Status Codes in Spring Boot

- ResponseEntity
  - Used only when the status code varies by runtime condition
  - In the example
    - ResponseEntity.created(location) sets 201 and the Location header in a single call
  - Use ResponseEntity when the status code or headers depend on the outcome of the operation, not for hardcoding a fixed code

```java
@PostMapping
public ResponseEntity<EmployeeDto> create(@Valid @RequestBody EmployeeCreateRequest
    EmployeeDto created = employeeService.create(req);
    URI location = URI.create("/api/v1/employees/" + created.id());
    return ResponseEntity.created(location).body(created);
}
```

© 2026 by Innovation In Software Corporation

Engineering Principles

Convention	Principle	Consequence of Violation
Use the correct 2xx code — 200, 201, or 204	Explicit signalling; uniform interface	Clients cannot distinguish a creation from a read. The `Location` header is lost. Clients make a second request to discover what was created.
Distinguish 401 from 403 precisely	Information security; least disclosure	401 when you mean 403 tells the client to re-authenticate when that will not help. 403 when you mean 401 skips the `WWW-Authenticate` header that clients need to initiate authentication.
Use 404 to prevent resource enumeration when existence is sensitive	Defence in depth	Returning 403 confirms the resource exists. An attacker can map your data by probing IDs and observing 403 vs 404 responses.
Distinguish 400 from 422	Correct client guidance	Returning 400 for a business rule violation tells the client to fix the request syntax, which is not the problem. The client wastes effort reformatting a request that was already correct.
Never return 200 with an error body	Separation of concerns; uniform interface	Infrastructure is blinded. Monitoring shows false success. Retry logic cannot function. Client libraries silently treat errors as successes.
Never return stack traces in 5xx responses	Information hiding; security by default	Stack traces expose class names, method signatures, framework versions, and database schema hints. Each is reconnaissance for an attacker.

© 2026 by Innovation In Software Corporation

Summary

- 401 and 403 are not interchangeable
  - 401 means authenticate
  - 403 means you are authenticated but not permitted
  - The client behaviour in response to each is different
  - The security implications of confusing them are real.
- 400 and 422 address different failure levels
  - 400 means the request cannot be understood
  - 422 means the request is understood but the domain state prevents it
  - The client needs to know which problem to fix

© 2026 by Innovation In Software Corporation

Summary

- The 200-with-error pattern is not a style preference
  - It is an architectural mistake that blinds infrastructure, corrupts metrics, and breaks retry logic
  - It must not appear in production APIs.
- All exception-to-status-code mappings belong in @ControllerAdvice
  - Controllers declare intent
  - The exception handler translates domain outcomes to HTTP outcomes
  - These concerns must not be mixed

© 2026 by Innovation In Software Corporation

Standard Error Payload Design

© 2026 by Innovation In Software Corporation

Error Payload Design

- Design issue
  - A consistent, machine-readable error contract is as important as the success contract
  - Clients must be able to handle errors programmatically, not by parsing strings

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

Error Payload Design

- When a call fails, an API client
  - Needs to know what category of failure occurred
    - Was the request wrong, did the resource not exist, did a business rule reject it?
  - Needs to know which specific fields were invalid so they can show the right error to the right user
  - Needs to be able to correlate the error with a server-side log entry when they file a support request
  - Needs to do all of this programmatically, in code, without parsing a human-readable text that might change across releases of the service

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

Error Payload Design

- If the error format is inconsistent
  - The consumer must write defensive error handling for every endpoint individually
  - Any change to an error message format breaks client code
  - Support requests cannot be correlated with server logs
  - Automated error classification becomes impossible
- The error contract is a contract
  - Once consumers depend on it, it has the same stability obligations as the success response
  - Most be designed before the first endpoint goes to production

Code	Name	When to Use
400	Bad Request	The request is syntactically invalid or structurally incorrect. Failed Bean Validation falls here. The server cannot understand the request as sent.
401	Unauthorized	Authentication is required but missing or invalid. The token is absent, expired, or has an invalid signature. The client must authenticate. The `WWW-Authenticate` response header must be included.
403	Forbidden	The client is authenticated but not authorised for this operation. The token is valid but the scope or role is insufficient. No `WWW-Authenticate` header — authentication is not the issue.
404	Not Found	The resource does not exist at this URI. Throw `EntityNotFoundException` in the service layer and map it to 404 in `@ControllerAdvice`.
409	Conflict	A state conflict prevents the operation. Duplicate resource creation against a unique constraint, or an optimistic lock violation where the client's version is stale. The client must resolve the conflict before retrying.
422	Unprocessable Entity	The request is syntactically valid and structurally correct, but violates a business rule. The JSON is well-formed and passes field validation, but the operation cannot be completed given the current state of the domain.
429	Too Many Requests	The client has exceeded a rate limit. Include a `Retry-After` header indicating when the client may try again.

© 2026 by Innovation In Software Corporation

Error Payload Design

- The recommended structure is aligned with RFC 7807
  - Problem Details for HTTP APIs
  - RFC 7807 is not mandatory
  - Following it provides a common convention
  - This is a best practice
- Suggest fields are shown on the right

Field	Type	Purpose
timestamp	ISO-8601 UTC string	When the error occurred. Format: `2024-11-15T14:32:00Z`. Enables chronological correlation across client and server logs.
status	Integer	The HTTP status code as an integer — e.g. `400`. Redundant with the status line but useful when the response is serialised to a log without headers.
error	String	A short, machine-readable error code in SCREAMING_SNAKE_CASE — e.g. `VALIDATION_FAILED`. Clients switch on this value. It must never change once consumers depend on it.
message	String	A human-readable description for developers. Not for end users. May be more verbose than the error code. This field can change wording without breaking client code because clients switch on `error`, not `message`.
path	String	The request URI that produced the error — e.g. `/api/v1/employees`. Useful for correlation when multiple endpoints share an error handler.
fieldErrors	Array	Present only on 400 validation errors. An array of objects, each containing `field`, `rejectedValue`, and `message`. Enables the client to surface per-field errors directly to the user.
traceId	String	A correlation identifier linking this error response to the full server-side log entry. Never a stack trace. Never an exception message. A trace ID only.

© 2026 by Innovation In Software Corporation

Payload Examples

- Validation failure 400

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "timestamp": "2024-11-15T14:32:00Z",
  "status": 400,
  "error": "VALIDATION_FAILED",
  "message": "Request validation failed. See fieldErrors for details.",
  "path": "/api/v1/employees",
  "fieldErrors": [
    {
      "field": "email",
      "rejectedValue": "not-an-email",
      "message": "must be a valid email address"
    },
    {
      "field": "salary",
      "rejectedValue": -1000,
      "message": "must be greater than 0"
    }
  ],
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

© 2026 by Innovation In Software Corporation

Payload Examples

- Resource not found 404

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "timestamp": "2024-11-15T14:32:01Z",
  "status": 404,
  "error": "EMPLOYEE_NOT_FOUND",
  "message": "Employee with id 42 does not exist",
  "path": "/api/v1/employees/42",
  "fieldErrors": [],
  "traceId": "9c1d3a8822e94f1b7d0c5e2f91a83b47"
}
```

© 2026 by Innovation In Software Corporation

Payload Examples

- Unhandled server error 500
- Notice what is absent from the 500 response
  - There is no exception class name
  - There is no database error message. There is no internal service name
  - The traceId is the only bridge between the client-facing response and the full server-side diagnostic information

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json

{
  "timestamp": "2024-11-15T14:32:02Z",
  "status": 500,
  "error": "INTERNAL_ERROR",
  "message": "An unexpected error occurred",
  "path": "/api/v1/employees",
  "fieldErrors": [],
  "traceId": "7e2b19c04d8a3f56e1907c4d82b5a631"
}
```

© 2026 by Innovation In Software Corporation

Error vs Message Field

- The error field is for machines
  - error is a stable, enumerated code in SCREAMING_SNAKE_CASE
  - Client code switches on this value to decide what to do
  - Because client code depends on it, it is part of the versioned API contract
  - Once a value is published and consumers are using it, it cannot change without a version increment

```
// Correct client-side error handling — switches on the stable code
if (apiError.error().equals("EMPLOYEE_NOT_FOUND")) {
    showNotFoundMessage();
} else if (apiError.error().equals("VALIDATION_FAILED")) {
    showFieldErrors(apiError.fieldErrors());
}
```

```
// Incorrect — switching on message text breaks when wording changes
if (error.message().contains("does not exist")) { ... }

// Correct — switching on the stable error code
if (error.error().equals("EMPLOYEE_NOT_FOUND")) { ... }
```

© 2026 by Innovation In Software Corporation

Error vs Message Field

- The message field is for developers
  - message is a human-readable description intended for developers reading logs or debugging integration
  - It can be more verbose, more contextual, and more explanatory than the error code
  - Crucially, its wording can change between releases without breaking any client code, because no well-written client switches on message text

```
// Correct client-side error handling — switches on the stable code
if (apiError.error().equals("EMPLOYEE_NOT_FOUND")) {
    showNotFoundMessage();
} else if (apiError.error().equals("VALIDATION_FAILED")) {
    showFieldErrors(apiError.fieldErrors());
}
```

```
// Incorrect — switching on message text breaks when wording changes
if (error.message().contains("does not exist")) { ... }

// Correct — switching on the stable error code
if (error.error().equals("EMPLOYEE_NOT_FOUND")) { ... }
```

© 2026 by Innovation In Software Corporation

Error vs Message Field

- The practical implication
  - Define the error code catalogue at the same time the API is designed
  - Treat the set of valid error values as a versioned enumeration
  - Document the error values
  - Once consumers depend on them, they should be as stable as the URL structure

```java
// Correct client-side error handling — switches on the stable code
if (apiError.error().equals("EMPLOYEE_NOT_FOUND")) {
    showNotFoundMessage();
} else if (apiError.error().equals("VALIDATION_FAILED")) {
    showFieldErrors(apiError.fieldErrors());
}
```

```java
// Incorrect — switching on message text breaks when wording changes
if (error.message().contains("does not exist")) { ... }

// Correct — switching on the stable error code
if (error.error().equals("EMPLOYEE_NOT_FOUND")) { ... }
```

© 2026 by Innovation In Software Corporation

The traceId Field

- The traceId field resolves a genuine tension in production error design
  - Operators need enough information to diagnose production errors quickly
    - A bare "An unexpected error occurred" message is useless when an operator is trying to find the relevant log entry among millions
  - Clients must never receive
    - Stack traces, internal class names, database error messages, SQL fragments, or any other implementation detail
    - Each of these is reconnaissance information
    - An attacker who receives a stack trace learns your framework versions, your internal package structure, your database driver, and potentially your schema

```
// Client logs the error for a support request
ERROR: API call failed — traceId: 4bf92f3577b34da6a3ce929d0e0e4736

// Operator queries the log aggregator
traceId=4bf92f3577b34da6a3ce929d0e0e4736

// Operator finds the full diagnostic entry
2024-11-15T14:32:00Z ERROR EmployeeService - EntityNotFoundException: Employee 42 not found
  at com.example.service.EmployeeService.findById(EmployeeService.java:47)
  ... full stack trace visible to operator only
```

© 2026 by Innovation In Software Corporation

The traceId Field

- The traceId is the bridge
  - The client receives a single correlation identifier
  - The operator queries the log aggregator for that identifier and finds
    - The full exception, stack trace, database query, timing information, and request context
  - Without any of that information having crossed the API boundary
    - The client gets enough to file a useful support request
    - The operator gets everything needed to diagnose the failure
    - Any potential attacker gets nothing useful

```
// Client logs the error for a support request
ERROR: API call failed — traceId: 4bf92f3577b34da6a3ce929d0e0e4736

// Operator queries the log aggregator
traceId=4bf92f3577b34da6a3ce929d0e0e4736

// Operator finds the full diagnostic entry
2024-11-15T14:32:00Z ERROR EmployeeService - EntityNotFoundException: Employee 42 not found
  at com.example.service.EmployeeService.findById(EmployeeService.java:47)
  ... full stack trace visible to operator only
```

© 2026 by Innovation In Software Corporation

The Error Code Catalogue

Error Code	Status	Meaning
`VALIDATION_FAILED`	400	One or more request fields failed Bean Validation. See `fieldErrors` .
`MALFORMED_REQUEST`	400	Request body could not be deserialised — invalid JSON or wrong content type.
`AUTHENTICATION_REQUIRED`	401	No token provided or token could not be validated.
`TOKEN_EXPIRED`	401	Token signature is valid but the token has expired.
`ACCESS_DENIED`	403	Token is valid but scope or role is insufficient for this operation.
`RESOURCE_NOT_FOUND`	404	The requested resource does not exist.
`EMPLOYEE_NOT_FOUND`	404	Domain-specific variant — use when the resource type adds clarity for the consumer.
`DUPLICATE_RESOURCE`	409	A resource with a conflicting unique identifier already exists.
`OPTIMISTIC_LOCK_FAILURE`	409	The resource was modified by another request since it was last read.
`BUSINESS_RULE_VIOLATION`	422	The request is structurally valid but violates a domain rule.
`RATE_LIMIT_EXCEEDED`	429	The caller has exceeded the permitted request rate.
`INTERNAL_ERROR`	500	An unexpected server error. No internal detail is provided. Use the traceId to correlate with server logs.
`SERVICE_UNAVAILABLE`	503	The server is temporarily unable to handle requests.

© 2026 by Innovation In Software Corporation

Engineering Principles

Convention	Principle	Consequence of Violation
Consistent error shape across all endpoints	Contract stability	Clients write per-endpoint error handling. Any shape change breaks client code. Error handling becomes a conditional forest that grows with every new endpoint.
Machine-readable `error` code, human-readable `message`	Separation of concerns	Clients switch on `message` text which changes between releases. Silent breakage when error descriptions are updated.
`traceId` in every error response	Information hiding combined with operability	Without a trace ID, operators cannot correlate a client-reported error with server logs. With internal detail in the response, attackers gain reconnaissance. The trace ID is the only acceptable bridge between these two needs.
`fieldErrors` array for validation failures	Correct client guidance	Without per-field errors, the client cannot tell the user which field to fix. The user sees a generic failure message and must guess.
Catch-all handler sanitises all 500 responses	Security by default	Without a catch-all, unexpected exceptions leak stack traces, class names, and database schema information to the client.
Error code catalogue versioned alongside the API	Backward compatibility	Renaming an error code after consumers depend on it is a silent breaking change. Client error handling breaks without any indication that the API changed.

© 2026 by Innovation In Software Corporation

Summary

- The error contract is a contract
  - Design it before the first endpoint ships
  - Every consumer who writes error handling against it has a dependency on its shape and stability
- The error field is for machines, the message field is for developers
  - Client code switches on error
  - The message can change wording freely
  - This separation is what allows the error description to improve over time without breaking client code
- The traceId resolves the tension between operability and security
  - Clients get a correlation handle
  - Operators get full diagnostic detail in the logs
  - Attackers get nothing about the server's internals

© 2026 by Innovation In Software Corporation

Summary

- The error code catalogue is versioned
  - Once consumers depend on an error code value, it cannot change within an API version
  - Define the catalogue deliberately and treat it with the same discipline as your URL structure and response schemas

© 2026 by Innovation In Software Corporation

API Versioning

© 2026 by Innovation In Software Corporation

API Versioning

- Every API will eventually need to change
  - Reflects inevitable changes in the domain
    - Fields get renamed
    - Types change
    - Business rules evolve
    - New requirements arrive that cannot be satisfied without altering the API contract
  - The problem
    - Once a client depends on an API, any breaking change to that API breaks that client
    - In a system with a limited of consumers this is manageable but does not scale well
    - In a system with external consumers you do not control, this is no longer manageable
  - Versioning
    - Preserve backward compatibility with existing clients while evolving the API
    - It creates a stable surface that consumers can depend on
    - Defines a separate surface where change can happen without disrupting them

© 2026 by Innovation In Software Corporation

API Versioning

- Adding versioning to an unversioned API after consumers are in production
  - This is a breaking change in itself
  - All clients that call /employees must be updated to call /api/v1/employees
  - Or you run both paths in parallel indefinitely
    - Means maintaining two routing structures with no clean deprecation path
- The cost of adding /api/v1/ from day one is zero
  - The cost of not adding it and needing it later is a forced migration of every consumer.

© 2026 by Innovation In Software Corporation

Breaking Change

- The versioning decision
  - Depends on a precise definition of what constitutes a breaking change
  - A change is breaking if any existing client can fail as a result of the change being deployed, without the client making any changes of its own

Change	Why It Breaks
Removing a field from the response	Clients reading that field receive null or fail deserialisation
Renaming a field	Equivalent to removing the old name and adding a new one
Changing a field type — string to integer	Client deserialisation fails
Making an optional request field required	Existing clients not sending the field receive 400
Changing a status code — 200 to 201	Clients that switch on status code break
Renaming an error code in the `error` field	Clients switching on error codes break silently
Broadening a permission requirement	Clients that previously succeeded now receive 403
Removing an endpoint	Clients calling it receive 404
Changing the meaning of an existing field	Clients interpret the value incorrectly

© 2026 by Innovation In Software Corporation

Non-Breaking Change

- The non-breaking list contains an important implication
  - Clients should be written defensively
  - Poor client error handling can turn a non-breaking server change into a client failure
  - For example, a client that
    - Throws an exception on an unknown JSON field
    - Exhaustively switches on an enum with no default case
  - This is often called client brittleness

Change	Why It Is Safe
Adding a new optional field to a response	Well-written clients ignore unknown fields — Jackson default
Adding a new optional field to a request	Existing clients simply do not send it
Adding a new endpoint	Existing clients are unaffected
Narrowing a permission requirement	Clients that previously succeeded continue to succeed
Adding a new valid value to an enumerated response field	Clients that handle unknown enum values gracefully are unaffected

© 2026 by Innovation In Software Corporation

Versioning Strategies

- Three approaches to API versioning
  - Strategy 1: URI Path Versioning
    - The version is part of the URL pat
    - Explicit, visible, and cacheable
    - No special client configuration required
    - Routing at the API gateway level is straightforward, route by path prefix
    - Easy to test in a browser
    - Visible in logs, dashboards, and access records without parsing headers

```
GET /api/v1/employees
GET /api/v2/employees
```

© 2026 by Innovation In Software Corporation

Versioning Strategies

- Three approaches to API versioning
  - Strategy 2: Accept Header Versioning
    - The version is carried in the Accept header as a media type
    - The same resource at the same URL, with a different representation requested via content negotiation
    - Invisible in URLs
    - Difficult to test in a browser or with simple tools
    - Requires all clients to set custom headers
    - Harder to route in API gateways
    - Harder to observe in access logs

```
GET /employees
Accept: application/vnd.company.api.v2+json
```

© 2026 by Innovation In Software Corporation

Versioning Strategies

- Three approaches to API versioning
  - Strategy 3 Query Parameter Versioning
    - The version is a query parameter
    - Easy to add to existing clients
    - Poorly cacheable because the version parameter pollutes the cache key inconsistently
    - Semantically incorrect because the version is not a filter on the resource, it is a routing instruction
    - Tends to be inconsistently applied across endpoints

```
GET /employees?api-version=2
```

Criterion	URI Path	Accept Header	Query Parameter
Explicit and visible	Yes — version in every URL	No — hidden in headers	Partial — visible but semantically wrong
Cacheable	Yes — URL is the cache key	Requires Vary header configuration	Inconsistent
Testable in a browser	Yes	No	Yes
Observable in logs	Yes — no header parsing needed	No — requires header extraction	Yes
API gateway routing	Trivial — route by path prefix	Requires header inspection	Possible but awkward
Client implementation cost	Zero — standard URL change	Custom Accept header on every call	Low
Technically RESTful	Pragmatically correct	Theoretically most correct	Questionable

© 2026 by Innovation In Software Corporation

Versioning

- Suggested workflow
- Step 1
  - All controllers are mapped under /api/v1/ from the first line of code
    - The version prefix is set at the class level on the @RequestMapping annotation
    - Every method in the controller inherits it automatically
    - There is no version logic inside method bodies. The version is an address, not a condition.
  - Adding /api/v1/ to a controller that previously had no version prefix
    - Is a breaking change for every client already integrated with the unversioned path

```java
@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeControllerV1 {
    // All methods inherit /api/v1/employees prefix
}
```

© 2026 by Innovation In Software Corporation

Versioning

- Step 2
  - Introduce /api/v2/ only when a breaking change is unavoidable
    - A v2 controller is a new class with a new @RequestMapping
    - It is not a modified v1 controller with version-branching logic inside it
    - No version branching logic inside a controller
- Step 3
  - Deprecate, do not delete
    - When v2 is live, v1 is deprecated
    - Deprecated does not mean removed
    - It means clients receive a signal that the version has a sunset date and should migrate

```java
// When a breaking change genuinely requires v2
@RestController
@RequestMapping("/api/v2/employees")
public class EmployeeControllerV2 {
    // Breaking change isolated here — v1 is untouched
}
```

© 2026 by Innovation In Software Corporation

Versioning

- Step 4
  - Document the deprecation with response headers
    - The Deprecation header signals that the endpoint is deprecated
    - The Sunset header communicates the date after which the endpoint will no longer be available
    - The Link header with rel="successor-version" points the consumer directly to the replacement
  - Well-written clients that inspect headers can surface these signals automatically

```
@GetMapping("/{id}")
public ResponseEntity<EmployeeDto> getById(@PathVariable Long id,
                                           HttpServletResponse response) {
    response.addHeader("Deprecation", "true");
    response.addHeader("Sunset", "Sat, 01 Mar 2026 00:00:00 GMT");
    response.addHeader("Link",
        "</api/v2/employees/{id}>; rel=\"successor-version\"");
    return ResponseEntity.ok(employeeService.findById(id));
}
```

© 2026 by Innovation In Software Corporation

The Version Increment Decision

- Changes that always require a version increment regardless of context
  - Removing any field from a response that consumers are known to read
  - Changing any field type in a response
  - Renaming any field in a response
  - Making any currently optional request field required
  - Changing the HTTP method for an existing endpoint
  - Changing the status code for a success response

© 2026 by Innovation In Software Corporation

The Version Increment Decision

- Changes that require analysis before deciding
  - Adding a new required field to a response
    - If consumers are well-written and ignore unknown fields, this is non-breaking
    - If any consumer uses strict deserialization that rejects unknown fields, it is breaking
    - When in doubt, treat it as non-breaking but communicate the addition in release notes and verify consumer behaviour before deploying
  - Changing validation constraints on an existing optional field
    - Making an existing constraint stricter, like tightening a `@Size` maximum, or changing a regex
    - Technically a breaking change for any client currently sending values that were valid before and are now rejected
    - Whether this requires a version increment depends on whether the tightening is a bug fix or a new requirement

© 2026 by Innovation In Software Corporation

The Version Increment Decision

- Changes that require analysis before deciding
  - Changing the set of valid values in an enumerated field
    - Adding a new value is non-breaking for clients that handle unknown enum values defensively
    - Removing a value is breaking for any client that sends or depends on that value
- The governance principle
  - Treat the breaking change question as a consumer impact question, not a technical question
  - Ask:
    - Which clients call this endpoint
    - What do they do with the response
    - Would this change cause any of them to behave incorrectly
    - If the answer to the last question is yes for any client, the change is breaking

© 2026 by Innovation In Software Corporation

Versioning and the Error Contract

- The error contract is subject to the same versioning rules as the success contract
- The error field values are part of the versioned contract
  - A client that switches on EMPLOYEE_NOT_FOUND will break silently if that code is renamed to RESOURCE_NOT_FOUND without a version increment
  - The client's error handling stops matching, the error is no longer handled specifically, and the application degrades without any obvious indication of what changed
  - The error code catalogue must be versioned alongside the success response schema
    - When a v2 endpoint is introduced, its error codes are defined as part of the v2 contract
    - If the v2 error codes differ from v1, both sets remain valid for their respective versions until v1 is retired
  - Adding new error codes within an existing version is non-breaking
    - A client that does not recognize a new error code falls through to its generic error handler, which should always exist
  - Removing or renaming existing error codes is breaking

© 2026 by Innovation In Software Corporation

Good Version Management

- A well-managed versioned API follows a predictable lifecycle for each version
- Phase 1: Active
  - The version is current
  - New features and non-breaking changes are added here
  - Consumers are encouraged to use this version for new integrations.
- Phase 2: Deprecated
  - A newer version exists and this version still work
  - The Deprecation and Sunset headers are present on all responses
  - Consumers are notified through release notes, changelogs, and API documentation
  - A migration guide from this version to the current version is published and maintained
  - The deprecation period should be long enough for consumers to migrate
    - For internal APIs with known consumers, Three to six months is typical
    - For external APIs with unknown consumers, twelve months is a more defensible minimum

© 2026 by Innovation In Software Corporation

Good Version Management

- A well-managed versioned API follows a predictable lifecycle for each version
- Phase 3: Sunset
  - The version returns 410 Gone on all endpoints rather than being removed silently
  - 410 is distinct from 404
    - it communicates that the resource existed and has been deliberately removed, not that it was never theer
    - Clients that have not migrated receive a clear signal rather than a mysterious 404 that could be mistaken for a path error
- Phase 4: Retired
  - After a further period at 410, the routing is removed entirely
  - The version is documented as retired in the API changelog.

© 2026 by Innovation In Software Corporation

Safe and Idempotent

- In a distributed system, network calls fail
  - The properties of safe and idempotent determine what you can safely do when they do
  - Not informal conventions or design preferences
  - Formal properties of HTTP methods defined in RFC 9110
    - Every HTTP client library, caching layer, load balancer, and API gateway is built on these definitions
  - Understanding them precisely is a prerequisite for
    - Writing correct retry logic
    - Designing safe infrastructure
    - Making defensible decisions about which HTTP method to use for a given operation

© 2026 by Innovation In Software Corporation

Idempotency and Safe Methods

© 2026 by Innovation In Software Corporation

Safe

- Safe
  - A method is safe if it has no observable side-effects on the server
  - Calling it does not change any server state
  - GET, HEAD, and OPTIONS are safe
  - Because a safe method cannot change state, clients and infrastructure can call them freely
    - A CDN can serve a cached GET response
    - A monitoring system can poll a health endpoint
    - A browser can pre-fetch a link
    - Caching is built on the safe property

Method	Safe	Idempotent	Reasoning
GET	Yes	Yes	No state change. Same result every time.
HEAD	Yes	Yes	No state change. Same result every time.
OPTIONS	Yes	Yes	No state change. Same result every time.
PUT	No	Yes	First call changes state. Subsequent identical calls produce the same state as the first.
DELETE	No	Yes	First call removes the resource. Subsequent calls find it already absent — same final state.
PATCH	No	Conditionally	Depends on patch semantics. See section 3.6.4.
POST	No	No	Each call may create a new resource. Duplicate calls produce duplicate resources.

© 2026 by Innovation In Software Corporation

Idempotent

- Idempotent
  - A method is idempotent if calling it multiple times produces the same server state as calling it once
  - The outcome of the first call and the tenth call are identical from the server's perspective
  - GET, PUT, DELETE, HEAD, and OPTIONS are idempotent but POST is not
  - Retry logic uses the idempotent property
    - When a network failure leaves the client uncertain whether the server received the request, retrying an idempotent method is safe
    - Retrying a non-idempotent method risks executing the operation twice

Method	Safe	Idempotent	Reasoning
GET	Yes	Yes	No state change. Same result every time.
HEAD	Yes	Yes	No state change. Same result every time.
OPTIONS	Yes	Yes	No state change. Same result every time.
PUT	No	Yes	First call changes state. Subsequent identical calls produce the same state as the first.
DELETE	No	Yes	First call removes the resource. Subsequent calls find it already absent — same final state.
PATCH	No	Conditionally	Depends on patch semantics. See section 3.6.4.
POST	No	No	Each call may create a new resource. Duplicate calls produce duplicate resources.

© 2026 by Innovation In Software Corporation

Idempotent

- In every distributed system, network calls fail routinely
  - Connections time out
  - Load balancers drop packets
  - Services restart mid-request
- The ambiguity problem
  - When a network call fails
    - Did the server receive the request and process it successfully, then fail before returning the response?
    - Or did the request never arrive at all?

Method	Safe	Idempotent	Reasoning
GET	Yes	Yes	No state change. Same result every time.
HEAD	Yes	Yes	No state change. Same result every time.
OPTIONS	Yes	Yes	No state change. Same result every time.
PUT	No	Yes	First call changes state. Subsequent identical calls produce the same state as the first.
DELETE	No	Yes	First call removes the resource. Subsequent calls find it already absent — same final state.
PATCH	No	Conditionally	Depends on patch semantics. See section 3.6.4.
POST	No	No	Each call may create a new resource. Duplicate calls produce duplicate resources.

© 2026 by Innovation In Software Corporation

Idempotent

- The ambiguity problem
  - For an idempotent method, this ambiguity does not matter
    - The client retries
    - If the server processed the first request, the retry produces the same state
    - If the first request never arrived, the retry is the first execution
  - For a non-idempotent POST, the ambiguity is critical
  - If the server processed the first request and the client retries, the operation executes twice

```
// The POST retry problem illustrated

Client sends:  POST /employees  { "name": "Alice" }
Network:       Timeout — client receives no response

Question:      Did the server create Alice?
Answer:        Unknown.

If Alice was created and client retries:   Two Alices exist.
If Alice was not created and client retries: One Alice exists.

The client cannot distinguish these cases from the error alone.
```

© 2026 by Innovation In Software Corporation

Idempotent

- Idempotency-key pattern for POST
  - The client generates a unique key before the first attempt
  - Sends the key with every retry of the same logical operation
  - The server stores the key alongside the result of the first successful execution
  - Returns the stored result for any subsequent request with the same key, without executing the operation again

```
// Step 1 — Client generates a unique key before the first attempt
// The same key is used for all retries of this operation

POST /employees
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{ "name": "Alice", "department": "Engineering" }

// Step 2 — Server processes the request and stores the result against the key

HTTP/1.1 201 Created
Location: /api/v1/employees/42
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// Step 3 — Client retries after a network failure, with the same key

POST /employees
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// Step 4 — Server recognises the key, returns stored result without re-executing

HTTP/1.1 201 Created           // Same response — not 409 Conflict
Location: /api/v1/employees/42 // Same location — same resource
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

© 2026 by Innovation In Software Corporation

Idempotent

- Idempotency-key pattern for POST
  - The key is generated by the client, not the server
    - The client must generate it before the first attempt and reuse it for every retry
    - A new key on each retry defeats the purpose entirely.
  - The server returns the original response on a duplicate, not 409 Conflict
    - The duplicate is not an error
    - It is a correctly handled retry.
    - The client receives the same 201 and the same Location header as if the first request had succeeded and been received

```
// Step 1 — Client generates a unique key before the first attempt
// The same key is used for all retries of this operation

POST /employees
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{ "name": "Alice", "department": "Engineering" }

// Step 2 — Server processes the request and stores the result against the key

HTTP/1.1 201 Created
Location: /api/v1/employees/42
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// Step 3 — Client retries after a network failure, with the same key

POST /employees
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// Step 4 — Server recognises the key, returns stored result without re-executing

HTTP/1.1 201 Created           // Same response — not 409 Conflict
Location: /api/v1/employees/42 // Same location — same resource
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

© 2026 by Innovation In Software Corporation

Idempotent

- Idempotency-key pattern for POST
  - The key must be globally unique
    - UUID v4 is the standard choice.
    - The server's key store must have a defined retention window long enough to cover any realistic retry window

```
// Step 1 — Client generates a unique key before the first attempt
// The same key is used for all retries of this operation

POST /employees
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{ "name": "Alice", "department": "Engineering" }

// Step 2 — Server processes the request and stores the result against the key

HTTP/1.1 201 Created
Location: /api/v1/employees/42
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// Step 3 — Client retries after a network failure, with the same key

POST /employees
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// Step 4 — Server recognises the key, returns stored result without re-executing

HTTP/1.1 201 Created           // Same response — not 409 Conflict
Location: /api/v1/employees/42 // Same location — same resource
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

© 2026 by Innovation In Software Corporation

Idempotent

- PUT is idempotent by specification
  - Two identical PUT requests must produce the same server state as one
  - PUT must replace the entire resource
    - Every field of the resource must be present in the request body
    - A field absent from the PUT body is not left unchanged but reset to null or its default value
    - This is what makes PUT idempotent
    - The server state after any number of identical PUT requests is fully determined by the contents of that request body, with no dependency on prior state

```
// PUT replaces the entire resource — missing fields reset to null

PUT /employees/42
{
  "firstName": "Alice",
  "lastName":  "Chen"
  // salary is absent
}

// Result: salary is set to null — not left at its previous value
// A second identical PUT produces the same result — idempotent
```

© 2026 by Innovation In Software Corporation

Idempotent

- PUT is idempotent by specification
  - RFC 7231 specifices that the PUT method replaces a resource rather than updating some selected fields
  - RFC 9110 specifies that after a PUT
    - A GET must return a representation equivalent to what was PUT
    - If absent fields from the put were silently preserved rather than cleared
    - A subsequent GET would return a resource that does not match what was PUT

"The PUT method requests that the state of the target resource be created or replaced with the state defined by the representation enclosed in the request message content."

"A successful PUT of a given representation would suggest that a subsequent GET on that same target resource will result in an equivalent representation being sent in a 200 OK response."

© 2026 by Innovation In Software Corporation

Idempotent

- PUT implementation mistake
  - Implementing PUT as a selective update is PATCH semantics
  - Only writing fields present in the body and leaving absent fields unchanged breaks idempotency
  - The outcome depends on the prior state of the resource, not solely on the request body
  - It misleads clients who expect PUT to be a full replacement
  - It creates subtle bugs when clients send a partial body expecting the absent fields to be cleared

Method	Semantics	Idempotency
PUT /employees/42	Replace entire resource. All fields must be present. Absent fields reset to null.	Idempotent — second identical PUT produces identical state
PATCH /employees/42	Partial update. Only specified fields change. Omitted fields are untouched.	Conditionally idempotent — depends on patch semantics
POST /employees	Create new resource. Server assigns the ID.	Not idempotent — duplicate POST creates duplicate resource

© 2026 by Innovation In Software Corporation

PATCH: Partial Update Strategies

- When a client needs to update a subset of a resource's fields without providing the full representation, PATCH is used
  - It requires a precise specification of which fields are being changed
  - Two strategies are in common use
- Explicit nullable DTO
  - The client sends a JSON object where
  - Null means do not change
  - Non-null value means update to this value
  - The service layer applies only the non-null fields

```java
// Request record — null signals "leave unchanged"
public record EmployeePatchRequest(
    @Size(max = 100) String firstName,   // null = do not change
    @Size(max = 100) String lastName,    // null = do not change
    @Email           String email,       // null = do not change
    @Positive   BigDecimal salary        // null = do not change
) {}

// Service layer applies only non-null fields
public EmployeeDto patch(Long id, EmployeePatchRequest patch) {
    Employee employee = repository.findById(id)
        .orElseThrow(() -> new EntityNotFoundException(...));

    if (patch.firstName() != null) employee.setFirstName(patch.firstName());
    if (patch.lastName()  != null) employee.setLastName(patch.lastName());
    if (patch.email()     != null) employee.setEmail(patch.email());
    if (patch.salary()    != null) employee.setSalary(patch.salary());

    return toDto(repository.save(employee));
}
```

© 2026 by Innovation In Software Corporation

PATCH: Partial Update Strategies

- Explicit nullable DTO
  - Simple, type-safe, and easy to validate with standard Bean Validation
  - Limitation
    - Client cannot explicitly set a field to null using this strategy, because null is the signal for do not change
    - For most business fields this is acceptable
    - For optional fields that legitimately need to be cleared, a different approach is required

```java
// Request record — null signals "leave unchanged"
public record EmployeePatchRequest(
    @Size(max = 100) String firstName,   // null = do not change
    @Size(max = 100) String lastName,    // null = do not change
    @Email           String email,       // null = do not change
    @Positive   BigDecimal salary        // null = do not change
) {}

// Service layer applies only non-null fields
public EmployeeDto patch(Long id, EmployeePatchRequest patch) {
    Employee employee = repository.findById(id)
        .orElseThrow(() -> new EntityNotFoundException(...));

    if (patch.firstName() != null) employee.setFirstName(patch.firstName());
    if (patch.lastName()  != null) employee.setLastName(patch.lastName());
    if (patch.email()     != null) employee.setEmail(patch.email());
    if (patch.salary()    != null) employee.setSalary(patch.salary());

    return toDto(repository.save(employee));
}
```

© 2026 by Innovation In Software Corporation

PATCH: Partial Update Strategies

- JSON Patch (RFC 6902)
  - The client sends an array of patch operations
  - Each operation specifies what to do, which field to do it to, and the value to use
  - Supports add, remove, replace, move, copy, and test operations
    - The remove operation allows explicitly clearing a field, which the nullable DTO strategy cannot do
  - More complex to implement and validate on the server side
  - Requires the application/json-patch+json content type

```
PATCH /employees/42
Content-Type: application/json-patch+json

[
  { "op": "replace", "path": "/salary",     "value": 95000 },
  { "op": "add",     "path": "/middleName", "value": "Jane" },
  { "op": "remove",  "path": "/nickname"                   }
]
```

© 2026 by Innovation In Software Corporation

PUT vs PATCH

- The choice between PUT and PATCH is not a preference
  - Follows from the nature of the client's relationship with the resource
- Use PUT when
  - The client always has the full resource state and replaces it atomically
  - The client fetched the resource, modified it in memory, and is writing the whole thing back
    - Configuration objects and settings records are the clearest examples
    - The client reads the full configuration, changes one setting, and writes the entire configuration back

```
// Natural PUT scenario — client has the full state
GET /employees/42/preferences → returns all 12 preference fields
Client changes notificationsEnabled from true to false
PUT /employees/42/preferences → sends all 12 fields back
```

```
// Natural PATCH scenario — client knows only what it changed
User updates their display name in a profile form
PATCH /employees/42 → sends only { "displayName": "Alice C." }
All other fields untouched — client never fetched them
```

© 2026 by Innovation In Software Corporation

PUT vs PATCH

- Use PATCH when
  - The client operates on a subset of the resource's fields
  - The full resource state is large, expensive to fetch, or unknown to the client
  - Different clients update different fields concurrently and per-field optimistic locking is appropriate
  - Mobile clients where bandwidth cost makes sending the full resource wasteful

```
// Natural PUT scenario — client has the full state
GET /employees/42/preferences → returns all 12 preference fields
Client changes notificationsEnabled from true to false
PUT /employees/42/preferences → sends all 12 fields back
```

```
// Natural PATCH scenario — client knows only what it changed
User updates their display name in a profile form
PATCH /employees/42 → sends only { "displayName": "Alice C." }
All other fields untouched — client never fetched them
```

© 2026 by Innovation In Software Corporation

PUT vs PATCH

- Start with PUT for simple resources
  - Introduce PATCH when partial update requirements emerge from actual usage patterns
  - Do not implement PATCH speculatively
- Mistake to avoid
  - Implementing PATCH as a full replacement
  - It behaves like PUT but is documented as PATCH, breaking both client expectations and idempotency reasoning
  - If a PATCH implementation silently nullifies fields the client did not include
    - It will cause data loss that is difficult to diagnose because the HTTP method suggests partial update semantics

```
// Natural PUT scenario — client has the full state
GET /employees/42/preferences → returns all 12 preference fields
Client changes notificationsEnabled from true to false
PUT /employees/42/preferences → sends all 12 fields back
```

```
// Natural PATCH scenario — client knows only what it changed
User updates their display name in a profile form
PATCH /employees/42 → sends only { "displayName": "Alice C." }
All other fields untouched — client never fetched them
```

© 2026 by Innovation In Software Corporation

Response Headers

- Status codes communicate outcome
- Response headers communicate metadata
  - Enables every other layer of the system to act on that outcome correctly
- The headers an API includes by default are an operability decision
  - Headers are read by infrastructure, monitoring tools, and well-written client libraries without any application-level code
  - Every header omitted is a capability removed from every layer that would have used it

```
// Natural PUT scenario — client has the full state
GET /employees/42/preferences → returns all 12 preference fields
Client changes notificationsEnabled from true to false
PUT /employees/42/preferences → sends all 12 fields back
```

```
// Natural PATCH scenario — client knows only what it changed
User updates their display name in a profile form
PATCH /employees/42 → sends only { "displayName": "Alice C." }
All other fields untouched — client never fetched them
```

© 2026 by Innovation In Software Corporation

Response Headers

Header	When to Include	Purpose
`Content-Type: application/json`	Always	Explicit content type. Include `charset=UTF-8` for non-ASCII content. Never leave it to the client to guess.
`Location: /api/v1/employees/42`	Always on 201 Created	Points to the newly created resource. Clients retrieve the new resource without a second lookup. Without this header, the client must re-query to discover what was created.
`ETag: "abc123"`	On GET responses for resources supporting conditional updates	Enables optimistic locking at the HTTP layer. Client sends `If-Match: "abc123"` with PUT or PATCH. Server rejects the update if the resource has been modified since the client read it.
`Cache-Control: no-store`	On all authenticated API responses	Prevents proxies and browser caches from storing responses that contain user-specific or sensitive data.
`X-Trace-Id: 4bf92f35...`	On all responses — success and error	Allows clients to include the trace ID in support requests. Correlates client-observed errors with server-side log entries.
`Deprecation: true`	On responses from deprecated endpoints	Machine-readable signal that this endpoint will be removed. Client libraries and API monitoring tools can surface this automatically.
`Sunset: Sat, 01 Mar 2026 00:00:00 GMT`	Paired with Deprecation	The specific date after which the endpoint will no longer be available. Gives clients a concrete migration deadline.

© 2026 by Innovation In Software Corporation

Questions?